A Hybrid Automata Framework for the Representation and Analysis of Microbial Co-Cultures

by

Jacob Francis Bowers

Bachelor of Science, Mathematical Sciences
Department of Mathematical Sciences
Florida Institute of Technology
2017

Bachelor of Science, Computer Science
Department of Computer Sciences & Engineering
Florida Institute of Technology
2017

A thesis
submitted to the College of Engineering and Science
at Florida Institute of Technology
in partial fulfillment of the requirements
for the degree of

Master of Science
in
Computer Science

Melbourne, Florida
May, 2021

© Copyright 2021 Jacob Francis Bowers

All Rights Reserved

The author grants permission to make single copies.

ABSTRACT

Title:

A Hybrid Automata Framework for the Representation and Analysis of Microbial Co-Cultures

Author:

Jacob Francis Bowers

Major Advisor:

Siddhartha Bhattacharyya, Ph.D.

Microbes can transform their environment in unpredictable ways with beneficial outcomes. Microbial species in nature are rarely found in isolation. Instead most of them form complex communities or consortia with other microbial species. The transformative potential of such microbial communities far exceeds that of any individual species. Microbial co-cultures grown under controlled laboratory environments or microbial consortia found in nature, by virtue of their own biology, can produce molecules with applications in multiple domains e.g. medicine, agriculture and bio-energy. Hence, a structured approach to the understanding of synergistic microbial capacity can significantly boost the utilization of microbial co-cultures/consortia for industrial applications. To this end, a modelling framework has been developed that can demonstrate the difference in metabolism between the individual mono-cultures and the co-culture of two simplistic *in silico* organisms. This framework can produce hybrid automata representations of an organism directly from mass balance, stoichiometry, and flux bounds. This approach is both generic with respect to the microbe(s) under consideration and scalable in the size of the relevant metabolic network(s). Constructing a reduction from the trajectory computed by dynamic flux balance analysis (dFBA) to a hybrid automaton with the same dynamical behavior comprises the core of the modelling effort. The transformation to hybrid automaton is achieved by basis vector decomposition of the

flux cone and the subsequent use of non-determinism to generically represent trajectories. Considerations from control theory are also taken into account. Using Ariadne, a C++ library for hybrid systems analysis, two in silico organisms are evolved in both mono-culture and co-culture, identifying emergent metabolic phenomena resembling the predictions given by dFBA. Furthermore, this framework provides a foundation to perform parametric analysis and identify unsafe regions.

Table of Contents

Abstract iii

List of Figures ix

List of Tables xi

Abbreviations xii

Acknowledgments xiii

Dedication xiv

1 Introduction 1

2 Systems Biology Preliminaries 5

2.1 Reaction Modelling Fundamentals 5

2.1.1 General-Case Dynamics 5

2.1.2 Michaelis-Menten Kinetics 5

2.1.3 Other Steady-State Reaction Mechanisms 5

2.2 Systems Biology Techniques 5

2.2.1 Flux Balance Analysis 5

2.2.2 Dynamic Flux Balance Analysis 7

2.2.3 Elementary Mode Analysis 8

2.2.4 Extreme Pathways Analysis 8
2.2.5 Metabolic Control Analysis 9

3 Hybrid Systems Preliminaries 10

3.1 Theoretical Foundations of Hybrid Systems 10
3.1.1 Formalism of a Hybrid System 10
3.1.2 Hybrid Systems as Hybrid Automata 11
3.1.3 Simulation, Bisimulation, and Languages over Automata 12
3.2 Formal Verification of Hybrid Automata 14
3.2.1 Hybrid Automata and Decidability 14
3.2.2 Taylor Models and Flowpipe Construction 15
3.2.3 Other Common Approaches 21
3.2.4 The Ariadne C++ Library 21
3.2.5 Theoretical Foundations of Ariadne 21
3.3 Hybrid Automata Graphical Notation 22

4 Literature Review 29

5 Metabolic Modelling with Hybrid Automata 37

5.0.1 Formulation of a Reference Model 37
5.1 Flux-Based Modelling Approaches 42
5.1.1 Decomposing Steady-State Metabolism 42
5.1.2 Representing Biological Growth 52
5.1.3 Incorporating Flux Bounds with Decomposed Dynamics 54
5.1.4 Replacing Nondeterminism with Deterministic Control 55
5.1.5 Co-Culture as Composition of Automata 58
5.1.6 Algorithmically Assembling a Model 60

6 Applying the Framework **63**

6.1 Initial Automata Models 63

6.1.1 Organism Automata 63

6.1.2 Deterministic Control of Model Organisms 66

6.1.3 Co-Culture Automaton 68

6.2 Simulation Environment Setup 69

6.2.1 Automata Implementation in Ariadne 69

6.2.2 Automata Variations 69

6.2.3 Simulation Parameters 69

6.2.4 Simulation Output, Data Gathering, and Plotting 69

6.3 Experimental Results 69

6.3.1 Reachable Set Computation 69

6.3.2 Qualitative Behavior 69

6.3.3 Formal Verification of Emergent Metabolism 70

7 Systems Biology Research with Hybrid Automata **71**

7.1 Extreme Pathways to Regulatory Mechanisms 73

7.1.1 Singular Value Decomposition and Eigenpathways 73

7.1.2 Nondeterministic Regulatory Dynamics 76

7.1.3 Formalizing Regulatory Queries 78

7.1.4 From Optimal Control to Regulatory Mechanisms 78

7.2 From Genome-Scale Models to Hybrid Automata 83

7.2.1 Sampling the Flux Space 84

7.2.2 A Genome-Scale Analytical Pipeline 84

7.3 Other Convex Basis Parameterizations 85

7.3.1 Elementary Mode Analysis 85

8 Conclusions and Future Work **86**

8.1 Conclusions 86

8.2 Future Work 87

8.2.1 Genome-Scale Model Analysis 87

8.2.2 Solving the Optimal Control Problem 87

8.2.3 Differential Dynamic Logic 87

References **88**

A Background **92**

List of Figures

3.1 23

3.2 23

3.3 24

3.4 24

3.5 25

3.6 25

3.7 26

3.8 26

3.9 27

3.10 27

5.1 Visualization of the co-culture formed by Organism 1 and Organism 2. Note the interaction between them by means of metabolite C, which Organism 1 processes to yield metabolites Y and Z. 40

5.2 Visualization of $\mathbb{H}(\mathbb{D}, \overline{V})$ for an arbitrary $\mathbb{D}(\overline{V})$ 45

5.3 Visualization of $\mathbb{H}(\mathbb{D})$ for an arbitrary $\mathbb{D}$ 52

5.4 Output of Algorithm 2 with Biomass Accumulation 53

5.5 Output of Algorithm 2 with Flux Bounds and Biomass Accumulation . 55

5.6 Adapting the Automaton to use Deterministic Transitions 57

6.1 Automata representation of Organism 1 65

6.2 Automata representation of Organism 2 66
6.3 Organism 1 with Control Mechanism 67
6.4 Organism 2 with Control Mechanism 67
6.5 Co-Culture with Control Mechanism 68
6.6 Levels of emergent metabolite Y in single cultures and co-culture . . . 70

7.1 General form of a microbe automaton using naive bases 73
7.2 Microbe automaton as parameterized by the eigenpathways 75
7.3 Microbe automaton as parameterized by E_k. Generally, $k << m$, rendering the model significantly more computationally efficient. 76
7.4 Microbe automaton with E_k parameterization and deterministic control functions. 79
7.5 Automaton representing cellular regulation as generally understood. Each regulatory mode can transition to any other regulatory mode. $k = 4$ has been chosen but k is arbitrary. Transitions to l_1 driven by $[\overline{Q}]_i > [v_{max}]_i * \Delta t$ and $[\overline{Q}]_i < [v_{min}]_i * \Delta t$ have been omitted for clarity but would be present in the model. 80
7.6 Model generation pipeline from genome-scale stochiometry. 84

List of Tables

List of Symbols, Nomenclature or Abbreviations

$\emptyset$	The empty set
$\mathbb{N}$	The set of all natural numbers
$\mathbb{R}$	The set of all real numbers
$\mathbb{R}^n$	The space of all real vectors of length n, $n \in \mathbb{N}$.
Some lowercase letter, e.g., x, v, b	A vector in some real space $\mathbb{R}^n$
Some uppercase letter, e.g., A, C, X, T	Either a real variable in $\mathbb{R}$, a set of real variables, a set of real vectors, or a matrix of real numbers
$\overline{X}$	A vectorization of the set of variables X
$[\overline{X}]_i$	The variable at element i in $\overline{X}$
$[v_1, v_2, ..., v_m]$	A vector of real vectors, e.g., each $v_i \in \mathbb{R}^n, 1 \leq i \leq m$
$x \cdot y$	The dot product between two vectors $x, y \in \mathbb{R}^n$
$Q \cdot v$	Matrix multiplication between $Q \in \mathbb{R}^{M \times N}$ and $v \in \mathbb{R}^n$
The lowercase letter t	The real-valued independent variable $t \in \mathbb{R}$
Some uppercase letter $\mathbb{T}, \mathbb{D}, \mathbb{H}$, etc.	A tuple of real variables, sets, vectors, vectorized sets of real variables, and matrices
$\bigcap$	\bigcap

Acknowledgements

This work was supported, in part, by Data Analysis Overhead Funding from NREL's Energy Systems Integration Facility and Florida Institute of Technology.

Dedication

Here is your dedication! Lorem ipsum dolor sit amet, idque apeirian mnesarchum vix ea, usu no mandamus constituam comprehensam, cum cu vero definitionem. Fabulas probatus ne pro, eleifend hendrerit adipiscing at vim. Usu eu facilisis persecuti. Ne has vidit principes efficiantur, has liber euripidis no. Vide sapientem salutandi per at. Eu probo democritum referrentur pri, an vidit simul reprimique nam. Omnium eruditi ex per, elit regione reprimique te eos. Dicam timeam no nam. Tacimates indoctum inciderint ad his, sea fastidii appetere oporteat an. Ad cum movet efficiantur. Sale labitur no vis, complectitur consequuntur usu te. Mel te putant timeam denique. Ne mel vidit ancillae. Ius saperet facilisi an. Eligendi scaevola sea ex, ad recteque cotidieque quo. Te sumo brute eum.

Chapter 1

Introduction

Microbes by themselves can produce chemical molecules that have significant use as industrial chemicals or their precursors [9, 7]. These single microbial species are limited in their metabolic capacity. However, the interactions between different microbial species in a microbial community can result in emergent metabolic behaviors that support the production of chemical species that can have multiple uses, e.g., aid the process of stabilizing the ecosystem, enrich agriculture [15] and maintain stable human health [17, 8, 24]. Presently, research efforts are beginning to focus on analysis and control of the metabolism of complex microbial communities [5, 23].

In the field of bioenergy, efficient consumption and conversion of both hexose and pentose sugar monomers in biomass hydrolysates is an important requirement for the commercialization of second-generation biofuels [12]. Biomass derived from plant matter is a heterogeneous material that yields a variety of pentose and hexose sugars when subjected to enzymatic hydrolysis [11, 30]. Microbial co-cultures/consortia have been shown to be more effective for fermentation of sugar mixtures to ethanol, compared to individual strains [18, 11]. Instead of attempting to engineer cells to deviate from their natural metabolic behavior, these consortia take advantage of the native capabilities of

different species to metabolize different sugars and direct available carbon to targeted biofuels or biofuel precursors. This strategy allows metabolic engineering efforts to focus on leveraging natural microbe behavior instead of introducing foreign genes whose effects are often counteracted upon [11, 16]. In other words, microbial mixed cultures allow the combination of the inherent conversion strengths of two or more microbes [10, 3].

The development of microbial consortia for efficient biofuel production is a very challenging problem. Experimental data on the behavior of microbial consortia is limited, especially for species combinations that do not appear naturally [6, 25, 28]. Moreover, purely experimental approaches are not well suited to handle the wide variability of available biomass feedstocks and the immense diversity of microbial species. Metabolic modeling offers the potential for the identification of synergistic combinations of biofuel-producing microbes and optimization of fermentation conditions for efficient substrate utilization and high product yields, thereby informing and guiding experimental efforts. Though there have been several approaches to create metabolic models for consortia [2], none of these approaches involve a well-defined formal method based approach. In this work, a hybrid systems approach is formulated to model metabolism in microbial co-cultures. This allows the introduction of formal methods from computer science/systems engineering to the realm of consortia bio-engineering.

For a metabolic network whose stoichiometry can be defined in sufficient detail, a number of analyses and simulation methods, termed *constrained-based metabolic modeling* can be brought to bear as reviewed by Chang et al [4]. The stoichiometry of the metabolic network encodes a great deal of information about the associated metabolism, including the network topology and allows only certain combinations of fluxes in the steady state. In the constrained-based metabolic modeling approach, a phenotypically feasible solution space is defined by specifying certain assumptions, e.g.,

a steady-state assumption [13], and the solution space can be further reduced by the imposition of constraints related to physico-chemical laws, genetics, and the environment [13, 21]. Thus, constraint-based techniques often focus on computing the possible steady states of a biochemical reaction network, given the stoichiometry of the network [29].

Most common among such analyses are methods like flux balance analysis (FBA) and its variants. Flux balance analysis (FBA) is a constraint-based method that maximizes a linear combination of reaction fluxes in the metabolic network, subject to a steady-state assumption, as well as lower and upper bounds on each reaction flux[20, 13]. FBA and its variants have usually been used to characterize metabolic interactions within a single microbe for applications in environmental and industrial biotechnology [23]. However, this approach can be extended to co-cultures of two or more microbes as described in Salimi et al. [26] and Perez-Garcia et al. [23].

By building on existing approaches in constraint-based metabolic modeling, particularly as applied to a simplistic microbial co-culture, a hybrid systems modeling framework for microbial co-cultures has been developed that preserves the phenotypically-defined structure of the solution space. In section **??**, the lack of hybrid systems approaches to model microbial co-cultures is detailed, followed by a description of constraint-based metabolic modeling methods such as FBA and dynamic FBA in section **??**. We then elaborate our approach in section **??** and discuss the results in section **??**.

Next, the hybrid systems modeling approach is formulated with respect to a simplified, "toy" system – namely, an *in silico* co-culture of two microbes with trivial metabolic networks (< 10 reactions each) that exhibit emergent metabolic behavior when placed in the same medium under specific iniial conditions. This toy system in particular is considered as it has been already used to show *in silico* emergent metabo-

lite production under the dynamic FBA approach[5]. Then, we discuss the model predicted behavior of the toy microbe co-culture in section **??**, followed by the modeling paradigm using hybrid systems approach discussed in section **??**. Next, we model the microbe interactions as hybrid systems discussed in section **??** and how our approach differs from previous approaches. Finally, we formalize the properties to check if they are satisfied by the modeled behavior. Our initial results, we see similar performance as elaborated by FBA analysis.

Chapter 2

Systems Biology Preliminaries

2.1 Reaction Modelling Fundamentals

2.1.1 General-Case Dynamics

2.1.2 Michaelis-Menten Kinetics

2.1.3 Other Steady-State Reaction Mechanisms

2.2 Systems Biology Techniques

2.2.1 Flux Balance Analysis

Flux balance analysis (FBA) presents a simple but powerful technique for calculating the steady-state fluxes for an organism, given the stoichiometry under constraints that follow from the metabolic network topology, the genetics of the organism, the properties of the growth medium and sometimes from physico-chemical laws. As a metabolic flux represents the rate of a corresponding reaction, there are usually bounds (ther-

modynamic, kinetic, or biological) on the rate at which the reaction can occur. The combinations of fluxes that satisfy the steady-state requirement while also respecting the established bounds form a convex polytope called the *flux cone* or *flux space*. Since an unlimited number of valid flux vectors exist within the flux cone (a specific assignment to each flux from the stoichiometric matrix should be called a *flux vector*), FBA seeks to find the flux vector that maximizes a given objective function. The objective function usually represents a biologically relevant property, like biomass production.

More formally, FBA describes a mixed-integer linear programming problem with the following conditions: With the internal stoichiometry of the organism given as the matrix $S \in \mathbb{R}^{M \times N}$, where each row represents the internal concentration of a metabolite $C_1, C_2, ...C_M$ and each column represents the value of a specific reaction/exchange flux, where one must solve for the flux vector $v = [R_1, R_2, ..., R_i, J_1, J_2, ...J_j] \in \mathbb{R}^N$ with respect to the bounds $v \leq v_{max}$ and $v_{min} \leq v$, given the objective function $G(v)$, such that:

$$Arg\ max\ G(v)$$

$$S \cdot v = 0$$

$$[v_{min}]_1 \leq R_1 \leq [v_{max}]_1$$

$$[v_{min}]_2 \leq R_2 \leq [v_{max}]_2$$

$$...$$

$$[v_{min}]_i \leq R_i \leq [v_{max}]_i$$

$$[v_{min}]_{i+1} \leq J_1 \leq [v_{max}]_{i+1}$$

$$[v_{min}]_{i+2} \leq J_2 \leq [v_{max}]_{i+2}$$

$$...$$

$$[v_{min}]_{i+j} \leq J_j \leq [v_{max}]_{i+j}$$

2.2.2 Dynamic Flux Balance Analysis

While FBA solves for the flux values that satisfy the biological constraints also maximizing the objective function, that alone is not enough to calculate the trajectory of an organism through time. FBA can be used to examine dynamic processes, e.g. microbial growth in batch cultures, by combining FBA with an iterative approach based on a quasi-steady-state assumption. This extension of FBA is known as static optimization–based dynamic FBA or simply dynamic FBA [1]. In this approach, the time horizon is divided into equally-spaced, discrete intervals of size Δt. FBA is used to solve for growth, nutrient uptake and by-product secretion rates at the beginning of

each time step. The quasi-steady state assumptions are made – these rates are assumed to be fixed for the duration of the time step – and the changes in the biomass and nutrient concentrations are calculated by integrating the concentration dynamics (as function of the rates) over the time-step. These updated concentrations are then used to calculate the maximum uptake rates for the next time step. This iterative procedure thus allows the simulation of batch experiments.

2.2.3 Elementary Mode Analysis

Elementary Mode Analysis is a computational tool to identify the structure of a metabolic network that links the cellular phenotype to the corresponding genotype (Trinh et al., Appl Microbiol Biotechnol. 2009 Jan; 81(5): 813–826.). The analysis can decompose an intricate metabolic network constituted of highly interconnected reactions into uniquely organized pathways. These pathways consisting of a minimal set of enzymes that can support steady state operation of cellular metabolism represent independent cellular physiological states. Such pathway definition provides a rigorous basis to systematically characterize cellular phenotypes, metabolic network regulation, robustness, and fragility that facilitate understanding of cell physiology and implementation of metabolic engineering strategies.

2.2.4 Extreme Pathways Analysis

Extreme pathways for a genome scale metabolic network constitute a unique and minimal set of vectors that completely characterize the steady-state capabilities of the network (Papin et al., Genome Res. 2002 Dec; 12(12): 1889–1900.). In other words, extreme pathways are mathematically derived vectors that can be used to characterize the phenotypic potential of a defined metabolic network (Schilling et al. 1999,

2000). Papin and co-workers have developed a framework to mathematically characterize extreme pathway length and to study how individual reactions participate in the extreme pathway structure of a network. These researchers have established that for any particular metabolic network, there is always a unique set of extreme pathways available (Papin, Jason; Nathan Price; Bernhard O Palsson (August 2004). "Comparison of network-based pathway analysis methods". Trends in Biotechnology. 22 (8): 400–405.). Moreover, a constraint-based approach has been developed in which constraints like mass balance and maximum reaction rates enable the demarcation of a 'solution space' where all the feasible options fall within. This constraint-based approach is supplemented by a kinetic modeling approach that enables the determination of a single solution in the extreme pathway solution space. This approach was initially applied to understand the human red blood cell metabolism. Thus, extreme pathway analysis can be used to understand the regulatory mechanisms of a metabolic network in further detail.

2.2.5 Metabolic Control Analysis

Metabolic Control Analysis (MCA) is a mathematical framework to determine, quantitatively, the degree of control that a given enzyme exerts on flux and on the concentration of metabolites, thus substituting the intuitive, qualitative concept of rate limiting step (Moreno-Sanchez et al., J Biomed Biotechnol. 2008; 2008: 597913.). Thus, MCA helps to understand the underlying mechanisms by which a given enzyme exerts high or low control and why the control of the pathway is shared by several pathway enzymes and transporters. By applying MCA it is possible to identify the steps that should be modified to achieve a successful alteration of flux or metabolite concentration in pathways of biotechnological importance e.g., large scale metabolite production.

Chapter 3

Hybrid Systems Preliminaries

3.1 Theoretical Foundations of Hybrid Systems

3.1.1 Formalism of a Hybrid System

Dynamical systems are mathematical models that capture the evolution of a set of variables over time (which can be continuous or discrete) with respect to given initial conditions. The set of variables, excluding time, collectively defines the state space of the system. Dynamical systems are traditionally classified into two categories, based on the nature of the state space – continuous dynamical systems and discrete dynamical systems. However, there is a category that subsumes both types of dynamical systems while also extending them – hybrid dynamical systems, or more commonly referred to as hybrid systems. Hybrid systems have a heterogeneous state space that contains both discrete and continuous variables, with complex evolution equations that make use of coupled discrete and continuous dynamics, non-determinism, and other constructs like differential inclusions or differential-algebraic equations. This generality makes hybrid systems capable of modeling a wide range of complex behaviors, but can often result

in ambiguous semantics and challenging analysis.

3.1.2 Hybrid Systems as Hybrid Automata

Hybrid automata are the conventional formalism for representing hybrid systems. Hybrid automata use a state machine representation, where discrete states (also referred to as *locations*) represent the set of discrete-valued variables. Transitions between locations represent a change in the value of the underlying discrete-valued variable. Each location is also endowed with a set of continuous variables and a vector field that defines the time evolution of the continuous set. Time is conventionally continuous-valued and the vector field is often an autonomous system (not explicitly dependent on time).

More formally, a hybrid automaton is a tuple $\mathbb{H} =$
$(Q, I, \Sigma, Y, F, T, G, R)$ consisting of the components:

State space: $Q = L \times X$,where L is a finite set of locations and X is the set of continuous variables. Each state can be described by $(l, x) \in Q$, where $l \in L$ and $x \in \mathbb{R}^n$.

Initial condition: $I = (l_0, x_0) \in Q$, the state of the system at time $t = 0$

Events: Σ is the finite alphabet or event set of $\mathbb{H}$. Let Σ always contain the *empty label* or ζ.

Outputs: Y is the output space of $\mathbb{H}$, which may consist of both continuous and discrete elements.

Continuous Dynamics: F is a function on L assigning a vector field or differential inclusion to each to each location in the automaton.

Transitions: $T \subset L \times \Sigma \times L$ is a set of directed edges. $t = (l, \sigma, l') \in T$ is a directed edge between a source location $l \in L$ and a target location $l' \in L$ with event label $\sigma \in \Sigma$. For any t with event label not explicitly given, let the event be labelled with the empty label ζ. In addition, $T = T_\Theta \cup T_U$, where T_Θ is the set of

forced transitions while T_U is the set of *unforced* transitions. Forced transitions take place instantaneously when the associated guard condition is crossed, while unforced transitions fire non-deterministically after the guard condition is crossed. That is to say, unforced transitions can take place at any time when the guard condition holds true, but are not required to. Unforced transitions may not fire at all.

Guard conditions: $G \subset 2^X \times T$ is the set of guard conditions for each transition $t \in T$. The shorthand $G(t) = g_t \in 2^X$ is used to associate a guard condition with its transition. If no g_t is explicitly given for some transition $t \in T$, then its default value is taken to be $X = R^n$.

Reset conditions: R is the set of reset conditions for each transition $t \in T$. The shorthand $R(t) = r_t$ is used, where $r_t : X \Rightarrow 2^X$ is a set-valued map. If no r_t is explicitly specified for some transition $t \in T$, then the default value is taken to be the identity function for all $x \in X$.

3.1.3 Simulation, Bisimulation, and Languages over Automata

Define the set of *trajectories* of some hybrid automata $\mathbb{H}$ as $\Omega(\mathbb{H})$. While trajectories are a well-formalized concept in the area of continuous dynamical systems, there are several conflicting definitions in the hybrid systems literature, so to be explicit, a trajectory of some hybrid automaton $\mathbb{H}$ is a *continuous trace* of the output space $\mathbb{H}_Y$. A continuous trace f of some variable x means that for each $t \in \mathbb{R}^+$, $f(x,t) = x_t$; that is, x has value x_t at time t. Note that there is no condition that the function f itself be continuous; hybrid automata often possess discontinuous jumps induced by resets, so $\lim_{t \to t_1} f(x,t) \neq f(x,t_1)$ upon some transition (and reset $x = c_1$) at time t_1. A continuous trace can also be defined for an arbitrary set of variables $X = \{x_1, x_2, ...\}$ such that $f(X,t) = \{f(x_1,t), f(x_2,t), ...\}$. Hence, a continuous trace of the output space $\mathbb{H}_Y$ is given by some function $f(\mathbb{H}_Y, t) = \{f(y_1,t), f(y_2,t), ...f(y_{|\mathbb{H}_Y|},t)\}$.

Two trajectories $f_1(\mathbb{H}_Y, t)$ and $f_2(\mathbb{H}_Y, t)$ are considered equivalent if and only if, for each $f_1(\mathbb{H}_Y, t_1)$ and $f_1(\mathbb{H}_Y, t_2)$ where $t_1 < t_2$, there exists a $t_1^{'}, t_2^{'} \in \mathbb{R}^+$ where $f_1(\mathbb{H}_Y, t_1) = f_2(\mathbb{H}_Y, t_1^{'})$ and $f_1(\mathbb{H}_Y, t_2) = f_2(\mathbb{H}_Y, t_2^{'})$ and $t_1^{'} < t_2^{'}$. Note that t_1, t_2 do not have to equal $t_1^{'}, t_2^{'}$ for the previous relation to hold; two trajectories are equivalent if both trajectories contain the same terms and the order in which the terms appear is the same. Hence, trajectories are *time-scale invariant*; $f(\mathbb{H}_Y, t)$ is equivalent to $f(\mathbb{H}_Y, ct)$ for all positive c under the previous relation.

Denote that two trajectories f_1, f_2 are equivalent by $f_1 \equiv f_2$. The relation $\equiv$ defines an equivalence class over the set of trajectories $\{f(X, t)\}$, where $X = \{x_1, x_2, ...\}$. Trajectories themselves can belong to the same equivalence class even if they originate from different hybrid automata, so long as $\mathbb{H}_{1_Y}$ and $\mathbb{H}_{2_Y}$ contain the same elements.

A hybrid automaton $\mathbb{H}_1$ is said to *simulate* another hybrid automaton $\mathbb{H}_2$ if $\mathbb{H}_{1_Y} = \mathbb{H}_{2_Y}$ and for each trajectory $f_2(\mathbb{H}_{2_Y}, t) \in \Omega(\mathbb{H}_2)$, there exists $f_1(\mathbb{H}_{1_Y}, t) \in \Omega(\mathbb{H}_1)$ such that $f_1(\mathbb{H}_{1_Y}, t) \equiv f_2(\mathbb{H}_{2_Y}, t)$. Note that this does not imply that $\mathbb{H}_{1_X} = \mathbb{H}_{2_X}$ or even that $\Omega(\mathbb{H}_2) \subset \Omega(\mathbb{H}_1)$. It simply states that every trajectory taken by $\mathbb{H}_2$ has an equivalent trajectory in $\Omega(\mathbb{H}_1)$ *under the output space* $\mathbb{H}_{1_Y} = \mathbb{H}_{2_Y}$. If the output space $\mathbb{H}_{1_Y} \subset \mathbb{H}_{1_X} + \mathbb{H}_{1_L}$ is "widened" to include more elements of $\mathbb{H}_{1_X} + \mathbb{H}_{1_L}$, $\mathbb{H}_1$ may not necessarily simulate $\mathbb{H}_2$ under this new shared output space. Denote that $\mathbb{H}_1$ simulates $\mathbb{H}_2$ by $\mathbb{H}_2 \subset \mathbb{H}_1$. If $\mathbb{H}_2 \subset \mathbb{H}_1$ and $\mathbb{H}_1 \subset \mathbb{H}_2$, they are said to *bisimulate*. Denote bisimulation by $\mathbb{H}_1 \equiv \mathbb{H}_2$. Bisimulation defines an equivalence relation (and hence equivalence classes) over the set of hybrid automata with the same output space. Denote the set of hybrid automata with some output space Y as $\{\mathbb{H}(Y)\}$. Hence, all hybrid automata belong to one (and only one) set of the form $\{\mathbb{H}(Y)\}$ and equivalence classes are defined over the set of all hybrid automata. From the above, an ontology of hybrid automata can be constructed. The set of all hybrid automata can be broken up into sets of hybrid automata $\{\mathbb{H}(Y_1)\}$, $\{\mathbb{H}(Y_2)\}, ...$ with output spaces $Y_1, Y_2, ...$ and

each set $\{\mathbb{H}(Y_i)\}$ is divided into equivalence classes by the bisimulation relation $\equiv$.

3.2 Formal Verification of Hybrid Automata

The formal verification of a hybrid automaton $\mathbb{H}$ involves checking the satisfaction of some predicate, p, over the set of continuous traces $\{f(\mathbb{H}_X + \mathbb{H}_L, t)\}$. The set of traces $\{f(\mathbb{H}_X + \mathbb{H}_L, t)\}$ is used instead of the set of trajectories $\Omega(\mathbb{H})$ as any predicate over the trajectories restricts p to the variables defined in the output space $\mathbb{H}_Y$. Invoking the operators of modal logic, p is a safety property if the satisfaction condition for p is $\Diamond p$, while p is a liveness property if the satisfaction condition for p is $\Box p$.

The term *formal verification*, as it will be used henceforth, refers to the computation of the *reachable set* of some $\mathbb{H}$ such that that the predicate $\Box p \mid \Diamond p$ is evaluated as $true$ or $false$. The reachable set of $\mathbb{H}$ is equivalent to $lim_{t\to\infty}\{f(\mathbb{H}_X + \mathbb{H}_L, t)\}$. The *reachability problem* for $\mathbb{H}$ is defined as $\exists f \in lim_{t\to\infty}\{f(\mathbb{H}_X + \mathbb{H}_L, t)\} \mid \Diamond p\ [f]$.

The *finite-time reachability problem* is of particular interest given the computational difficulty of precisely computing the reachable set for the general class of hybrid automata. The finite-time reachable set is given as $\Big\{\{f(\mathbb{H}_X + \mathbb{H}_L, t)\},\ \forall t \in [0, t_{\text{end}}]\Big\}$ and the associated reachability problem is defined as $\exists f \in \Big\{\{f(\mathbb{H}_X + \mathbb{H}_L, t)\},\ \forall t \in [0, t_{\text{end}}]\Big\} \mid \Diamond p\ [f]$.

3.2.1 Hybrid Automata and Decidability

The reachability problem is exactly decidable for a small subclass of hybrid automata. A precise boundary between decidable automata and undecidable was theoretically established by Henzinger et al. Furthermore, the results of Collins and Lygeros showed that the finite-time reachable set cannot always be approximated to an arbitrary level of precision. Hence, numerically-sound (under/over)-approximation techniques for reach-

able set computation must be developed to provide answers to the finite-time reachability problem for the general class of hybrid automata.

3.2.2 Taylor Models and Flowpipe Construction

One of the more promising techniques for finite-time reachable set computation involves approximating the automaton dynamics with *Taylor models*. Taylor models are a robust and generalizable framework for approximating a function by its Taylor series coupled with a symbolic remainder interval. They are useful in particular for verifying hybrid automata as there are few numerically-sound techniques for doing so when the automata in question contain nonlinear dynamics. Approximating a function with a Taylor model of degree k requires the function to be continuous and k-times differentiable. Conventional operations on functions, like addition, subtraction, composition, and integration are preserved under Taylor model approximation. The working definition of Taylor models and the associated operations in the elucidation to follow is provided by Berz and Makino, as expounded on by Chen et al. The theory of flowpipe construction is provided by the works of Chen et al. This section is intended to provide a high-level overview of the theory of Taylor models using the canonical definition that is familiar to researchers working in the area. Divergent formulations of Taylor models and deviations from the theory will be noted when they arise in the sections to come.

The Taylor approximation of a function $f,\ D \to \mathbb{R},\ D \subset \mathbb{R}^n$ with degree k is given as the polynomial p where $p(x_*) = f(x_*)$ for the central point $x_* \in D$ and for $1 \leq j \leq k,\ 1 \leq i \leq n,\ \partial^j f(x_*)/\partial x_i^j = \partial^j p(x_*)/\partial x_i^j$. A necessary and sufficient condition for the existence of p is that all partial derivatives of f with degree k exist and are themselves continuous, denoted by $f \in \mathbb{C}^k$. For any $f, g \in \mathbb{C}^k,\ f \equiv_k g$ iff there exists a polynomial p that is the Taylor approximation of both f and g. Hence, the Taylor approximation partitions $\mathbb{C}^k$ into a collection of equivalence classes.

Let $\mathbb{I}$ denote the set of all intervals $[l, u],\ l \leq u,\ l, u \in \mathbb{R}$. Intervals in $\mathbb{R}^n$ are given as hyper-rectangles and are elements of the set $\mathbb{I}^n = \mathbb{I} \times \mathbb{I} \times ... \times \mathbb{I} = \prod_{i=1}^{n} \mathbb{I}$.

A Taylor model (TM) of degree k over the domain $D \subset \mathbb{R}^n$ is a 2-tuple (p, I) where p is a polynomial of degree k over n variables and I is a remainder interval $I \in \mathbb{I}$. Denote a Taylor model with degree k of the function $f,\ D \to \mathbb{R}$ by $TM(f, k) = (p, I)$ where p is the Taylor approximation of f with degree k. Some (p, I) is a *bounded over-approximation* $f \in \mathbb{C}^k$ wherein $\forall x \in D, f(x) \in p(x) + I = \{p(x) + y \mid y \in I\}$.

As polynomials can be generalized to vectors of polynomials to approximate vector-valued real functions of the form $f,\ D \to \mathbb{R}^m$, Taylor models can be constructed for functions f insofar as they remain k-times differentiable for all partial derivatives. A k-degree Taylor model of f is given as $TM(f, k) = (p, I)$ where $I \in \mathbb{I}^m$.

To find the trajectory $\overline{X}(t)$ for some $d\overline{X}/dt = F(\overline{X}, t)$ with respect to the initial condition x_0, the *flow map* ϕ should be computed wherein $\overline{X}(t) = \phi(x_0)(t)$. The flow map defines the set of trajectories reachable from $x_0 \in X_0$ at time t and $\{\phi(x_0, t'),\ \forall x_0 \in X_0,\ \forall t' \in [0, t]\}$ is the reachable set at time t. However, many $d\overline{X}/dt$ do not have a closed-form solution ϕ that can be directly solved for. Hence, the calculation (and representation) of ϕ is an ideal domain for the application of Taylor models. For some dynamical system $f = d\overline{X}/dt$, denote the degree k Taylor model of its flow map ϕ_f by $FM(f, k) = TM(\phi_f, k) = (p, I)$, where p is the degree k polynomial and I is the symbolic interval of the Taylor model, both satisfying the necessary differentiability and over-approximation conditions with respect to ϕ_f.

There are two primary approaches to compute the polynomial p in $FM(f, k)$ for $f \in \mathbb{C}^k$. The first uses finite iterations (in fact, at most k are needed) of the *Picard operator* with the polynomial p as its fixed point.

Given some differential equation $d\overline{X}/dt = F(\overline{X}, t)$ and a function $g(x_0, t)$ where

$x_0 \in \overline{X}$, the Picard operator is defined as:

$$\mathbb{P}_F(g)(x_0, t) = x_0 + \int_{s=0}^{t} F(g(x_0, s), s)ds$$

If F is Lipschitz continuous, $\mathbb{P}_F$ has a unique fixed point g_* that is the solution $\overline{X}(t)$ to $d\overline{X}/dt = F(\overline{X}, t)$ with initial condition x_0 and successive iterations of the Picard operator are guaranteed to converge to g_*. More rigorously:

$$\mathbb{P}_F(\mathbb{P}_F ... \mathbb{P}_F(\mathbb{P}_F(g)))(x_0, t) = \lim_{n\to\infty} \mathbb{P}_F^n(g)(x_0, t) = g_*$$

To find the polynomial p for $FM(f, k)$, truncated iterations of the Picard operator $\mathbb{P}_f$ are applied such that the terms of degree greater than k are dropped after each iteration. This process is guaranteed to converge, given the convergence of the non-truncated Picard iterations when $F = f$.

As an example, take the differential equation $dx/dt = x^3 + 1$ with initial condition $x_0 = 1$. This is known to have closed-form solution $t + \frac{1}{18}(\sqrt{3}\pi + 6\ \log(2)) = \frac{tan^{-1}(\frac{2x(t)-1}{\sqrt{3}})}{\sqrt{3}} + \frac{1}{3}\log(1 + x(t)) - \frac{1}{6}\log(x(t)^2 - x(t) + 1)$. This is challenging to compute even with prior knowledge of its form and unwieldly to work with because of the difficulty of isolating $x(t)$. Hence, this is an excellent candidate to showcase the power of Taylor model integration. The polynomial p for the Taylor model $FM(x^3 + 1, 3)$ will be calculated. To calculate the first Picard iterate, set $g_0 = 1$ and $F = x^3 + 1$. Then, $\mathbb{P}_F(g_0)(1, t) = 1 + \int_{t=0}^{t}(1^3 + 1)\ dt = 2t + 1$. $\mathbb{P}_F^2(g_0)(1, t) = \mathbb{P}_F(g_1)(1, t) = 1 + \int_{t=0}^{t}((2t + 1)^3 + 1)\ dt = 1 + 2t + 3t^2 + 4t^3 + 2t^4$. This process is iterated until the truncated iterates converge, as demonstrated in the table below:

Convergence is achieved upon the third iteration and $p = 7t^3 + 3t^2 + 2t + 1$ for $p \in FM(x^3 + 1, 3)$. For the special case that f contains nonpolynomial terms, e.g. $FM(\frac{cos(x)}{e^x}, 5)$, an approximate polynomial term should be used to replace them before

	Picard iterate	Expanded terms
$g_1 = \mathbb{P}_F(g_0)(1,t)$	$1 + \int_{t=0}^{t}(g_0^3+1)\ dt$	2t + 1
$g_2 = \mathbb{P}_F^2(g_0)(1,t)$	$1 + \int_{t=0}^{t}(g_1^3+1)\ dt$	$2t^4 + \mathbf{4t^3} + \mathbf{3t^2} + \mathbf{2t} + \mathbf{1}$
$g_3 = \mathbb{P}_F^3(g_0)(1,t)$	$1 + \int_{t=0}^{t}(g_2^3+1)\ dt$	$\frac{8t^{13}}{13}+4t^{12}+12t^{11}+\frac{116t^{10}}{5}+34t^9+\frac{81t^8}{2}+\frac{279t^7}{7}+33t^6+\frac{117t^5}{5}+14t^4+\mathbf{7t^3}+\mathbf{3t^2}+\mathbf{2t}+\mathbf{1}$
$g_4 = \mathbb{P}_F^4(g_0)(1,t)$	$1 + \int_{t=0}^{t}(g_3^3+1)\ dt$	$\frac{64t^{40}}{10985}+\frac{256t^{39}}{2197}+\frac{192t^{38}}{169}+\ ...\ +\frac{4887t^8}{14}+\frac{6129t^7}{35}+\frac{837t^6}{10}+\frac{189t^5}{5}+\frac{65t^4}{4}+\mathbf{7t^3}+\mathbf{3t^2}+\mathbf{2t}+\mathbf{1}$

the Picard iterates are computed. In the previous scenario, $cos(x)$ and e^x would be replaced with their own Taylor polynomials, which then allow $\mathbb{P}_f(g_0)(x_0,t)$ to be computed as an integral of polynomial terms.

The second approach to compute the polynomial p for a flowmap uses a truncated *Lie series* built from the *Lie derivatives* $\mathcal{L}, \mathcal{L}^2, ..., \mathcal{L}^k$. The Lie derivative of some function $g(\overline{X},t)$ is $\mathcal{L}_F(g)(\overline{X},t) = \partial g(\overline{X}(t),t)/\partial t$ where $\overline{X}(t)$ is some trajectory and $d\overline{X}/dt = F(\overline{X},t)$. The Lie derivative is given by:

$$\mathcal{L}_F(g) = (\nabla_{\overline{X}}\ g)\cdot F(\overline{X},t) + \partial_t\ g = (\nabla_{\overline{X}}\ g)\cdot d\overline{X}/dt + \partial_t\ g$$

For some dynamical system $d\overline{X}/dt = F(\overline{X},t)$, a locally unique solution $\overline{X}(t)$ from the initial condition x_0 is given by $\overline{X}(t) = \phi(t,x_0)$, where $\phi(t,s) = \lim_{k\to\infty}\Sigma_{i=0}^{k}\frac{t^i}{i!}\mathcal{L}_f^i(x)(s,0)$.

Assembling the previous, the polynomial p for some $FM(f,k)$ is the truncated Lie series around $(x_0,0)$:

$$p = \Sigma_{i=0}^{k}\frac{t^i}{i!}\mathcal{L}_f^i(x)(x_0,0) = x_0 + t\mathcal{L}_f(x)(x_0,0) + \frac{t^2}{2}\mathcal{L}_f^2(x)(x_0,0) + ... + \frac{t^k}{k!}\mathcal{L}_f^k(x)(x_0,0)$$

As before, $p \in FM(x^3+1,3)$ will be computed as an example:

	Lie derivative
$\mathcal{L}_f(x)$	$1+x^3$
$\mathcal{L}_f^2(x) = \mathcal{L}_f(1+x^3)$	$3x^2+3x^5$
$\mathcal{L}_f^3(x) = \mathcal{L}_f(3x^2+3x^5)$	$6x+21x^4+15x^7$
$\mathcal{L}_f^4(x) = \mathcal{L}_f(6x+21x^4+15x^7)$	$6+84x^3+105x^6+105x^9$

The first four Lie derivatives are assembled into a Lie series as follows:
$\Sigma_{i=0}^4 \frac{t^i}{i!}\mathcal{L}_f^i(x) = x+(1+x^3)t+(3x^2+3x^5)\frac{t^2}{2}+(6x+21x^4+15x^7)\frac{t^3}{3!}+(6+84x^3+105x^6+105x^9)\frac{t^4}{4!} = t+\frac{t^4}{4}+x+t^3x+\frac{3t^2x^2}{2}+tx^3+\frac{7t^4x^3}{2}+\frac{7t^3x^4}{2}+\frac{3t^2x^5}{2}+\frac{35t^4x^6}{8}+\frac{5t^3x^7}{2}+\frac{35t^4x^9}{8}$.

This series is then evaluated at $x = x_0 = 1$, rendering $\mathbf{1+2t+3t^2+7t^3}+(25t^4)/2$. None of the $\mathcal{L}(x)...\mathcal{L}^4(x)$ need be evaluated at zero as they each contain no t terms due to the autonomous differential equation. Note that the terms with degree 3 or less are the same as those received from the Picard iterations. Hence, $p = 1+2t+3t^2+7t^3$.

In the case that x in $\mathcal{L}_f(x)$ is vector-valued, e.g. $x = \overline{X}$, m polynomials $p_1, ..., p_m$ must be calculated wherein each Lie series is respectively built with the Lie derivatives $\mathcal{L}_f(\overline{X}_j),\ 1 \leq j \leq m$, giving the solution $\overline{X}(t) = [p_1(t), ..., p_m(t)]$. This is because the gradient operator $\nabla_{\overline{X}}$ is not defined for a $g = x$ that is already vector-valued.

The second part of computing the Taylor model for a flowmap $FM(f,k)$ involves computing the symbolic interval I. To review, the interval I should be established such that $p(x_0,t) - \phi_f(x_0,t) \in I, \forall t \in [0,\Delta]$. The interval I can be computed by guessing a sample I_0 and applying the Picard operator on $p' = p + I_0$ to obtain p''. If $p'' - p \subset I_0$, I_0 is a valid interval and $FM(f,k) = (p,I)$ is a valid overapproximation of $\phi_f(x_0,t)$. This relationship holds because $p'' - p \subset I_0$ means the Picard operator is *contractive* on I_0. The interval can be tightened by repeatedly applying the Picard operator to achieve a series of $p'', p''', ..., p^n$ until the relationship $p_n - p \subset I_n$ no longer holds. In

the event that $p'' - p \not\subset I_0$, a bigger initial I must be chosen such that $I_0 \subset I_1$.

Using the theory of flowmaps, an overapproximation of the reachable set for the continuous system $d\overline{X}/dt = F(\overline{X}, t)$ can be computed as follows, with respect to the initial set X_0 and total time t:

1. Pick some interval $[0, \delta_1] \subset [0, t]$
2. Calculate the k-degree flowmap $FM(d\overline{X}/dt, k)$ with respect to the initial condition $x_0 \in X_0$
3. Compute the reachable set $X_1 = \{p(x_0)(t) + I,\ \forall t \in [0, \delta],\ \forall x_0 \in X_0\}$
4. Pick another interval $[0, \delta_2]$ such that $[0, \delta_1 + \delta_2] \subset [0, t]$
5. Repeat steps 2-3 over this interval to achieve the set X_2
6. Pick an interval $[0, \delta_i]$ where $[0, \delta_1 + ... + \delta_{i-1} + \delta_i] \subset [0, t]$ and iterate steps $2 - 3$ n times until $\delta_1 + ... + \delta_{n-1} + \delta_n = t$ and set X_n is achieved
7. Set X_n is the reachable set from X_0 over $[0, t]$

Smaller intervals $\delta_1, ..., \delta_n$ and higher degree Taylor models will result in a more accurate approximation of the reachable set. The size of the interval may be fixed $\delta_1, ..., \delta_n = \delta$ or may be determined on-the-fly within each iteration, with considerations made to the specific dynamic properties of the interval. Taylor model terms of high degree may be dropped between iterations as is often necessary to keep the overall degree of the approximation within a manageable upper bound.

It is important to note that Taylor models map the domain to a set of bounded regions in the codomain. Hence, Taylor models are able to provide a bounded overapproximation of the range of a wide variety of functions, and as shown before, can be used to provide guaranteed enclosures to the solution of most ordinary differential

equations. In practice, they have been shown to produce sound results for the trajectories of complex dynamical systems and do not lose numerically stability across long time-scales. Hence, Taylor models serve as a strong foundation for computing reachable sets for the general class of hybrid automata.

However, the integration of Taylor models alone is not enough to simulate the trajectories of hybrid automata. Specifically, the intersection of the integrated Taylor models (henceforth referred to as the *flowpipe*) and the guard conditions must be calculated in order to determine transition times.

Taylor models and guard conditions

3.2.3 Other Common Approaches

NuXMV

Hybrid automata with affine approximations

Differential dynamic logic

3.2.4 The Ariadne C++ Library

3.2.5 Theoretical Foundations of Ariadne

Ariadne is based on the theory of *computable analysis*, which seeks to provide a rigorous foundation for functions of a real-variable within a discrete, circuit-level representation. As the reals $\mathbb{R}$, their positive restriction $\mathbb{R}^+$ and the real vector spaces $\mathbb{R}^n, n \in \mathbb{N}$ have continuum cardinality, there is no direct way to map their structure into a discrete interval, such as that given by the 32 or 64-bit floating point numbers. The naive approach – taking a finitization of the uncountable set such that it maps onto the discrete structure – is not mathematically rigorous under the standard operations over the reals. A symbolic representation, then, may seem desirable, that avoids the truncation errors

induced by the previous approach, but symbolic operations quickly become memory-and-CPU intensive and unwieldly for serious computation. The theory of *computable analysis* seeks to remedy both of these issues by providing rigorous representations of real numbers and their functions without sacrificing computational efficiency.

Computable analysis represents the set $\mathbb{R}$ with a set $\mathbb{R}'$, a countable set of subsets of $\mathbb{R}$. Some element $x \in \mathbb{R}$ is then represented by $x' \in \mathbb{R}'$, where $x \in x'$. Each x' has a unique representation with a finite amount of data, such that $\mathbb{R}'$ can be mapped onto a discrete interval (e.g. a 32-bit register). Under the arithmetic operations $\{+, -, \times\}$, $x_1 \in x_1' \wedge x_2 \in x_2' \implies x_1 \oplus x_2 \in x_1' \oplus x_2', \forall x_1, x_2 \in \mathbb{R},\ x_1', x_2' \in \mathbb{R}',\ \oplus \in \{+, -, \times\}$. Other uncountable sets (including extensions of $\mathbb{R}$ like the complex numbers) and their operations can be defined similarly.

A function is *computable* insofar as it can be represented as a sequence of operations on a Turing machine.

Moore's interval arithmetic

Taylor models (already discussed)

Automatic differentiation

Constraint propagation

3.3 Hybrid Automata Graphical Notation

Like finite-state machines, infinite-state machines are most naturally expressed and understood in a graphical representation. In the case of hybrid automata, the discrete states (locations) are represented by nodes and transitions between discrete states (locations) are directed edges between nodes.

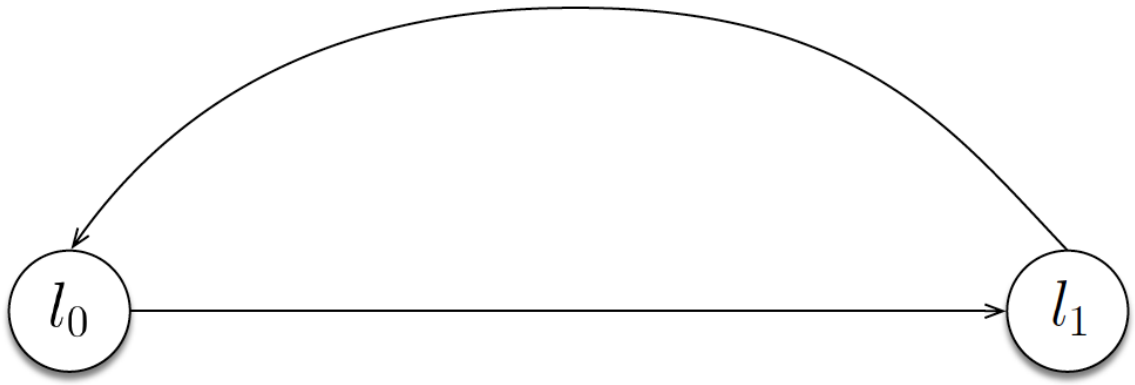


Figure 3.1

The initial location l_0 has a bold border; the initial condition x_0 will not be given in the diagram as multiple parameterizations of x_0 may be of interest. Hence, the diagram is representing a unique *system* but not necessarily one specific instantiation of the system.

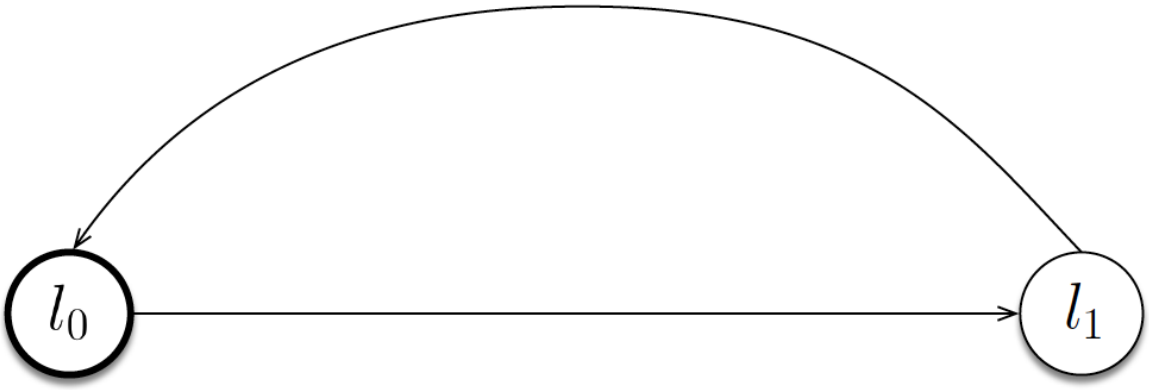


Figure 3.2

Forced transitions are represented with solid lines while dotted lines indicate unforced transitions as shown in Figure 3.3.

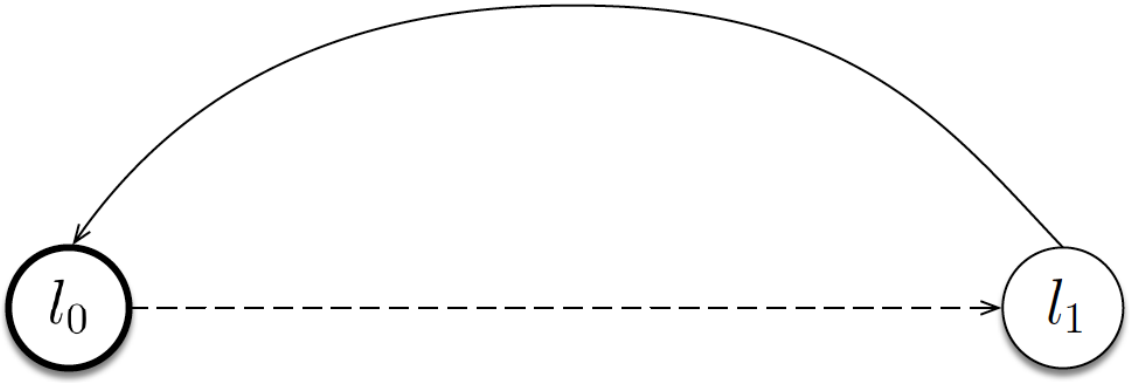
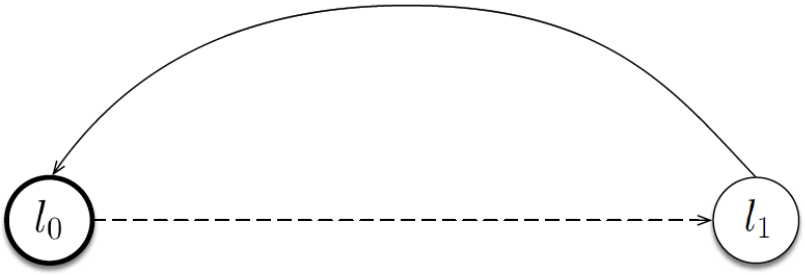


Figure 3.3

The vector field associated with a location is given within a box adjacent to the location itself. It can be given in vectorized or component-wise form:



$$\begin{bmatrix} \frac{da}{dt} = 1 \\ \frac{db}{dt} = 3b \\ \frac{dc}{dt} = 2t^2 + 2 \end{bmatrix} \qquad \begin{bmatrix} \frac{da}{dt} = 2t \\ \frac{db}{dt} = b^2 + 10 \\ \frac{dc}{dt} = a^3 \end{bmatrix}$$

Figure 3.4

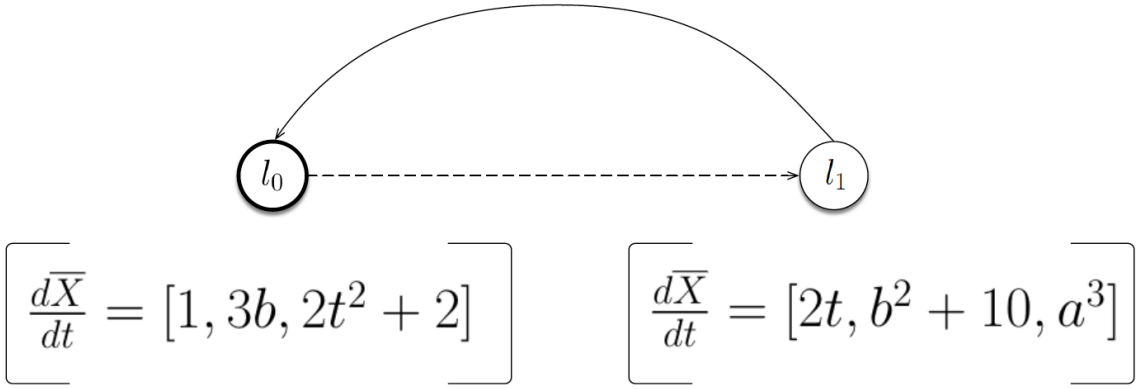


Figure 3.5

Vectorized notation may also be used in conjunction with component-wise notation:

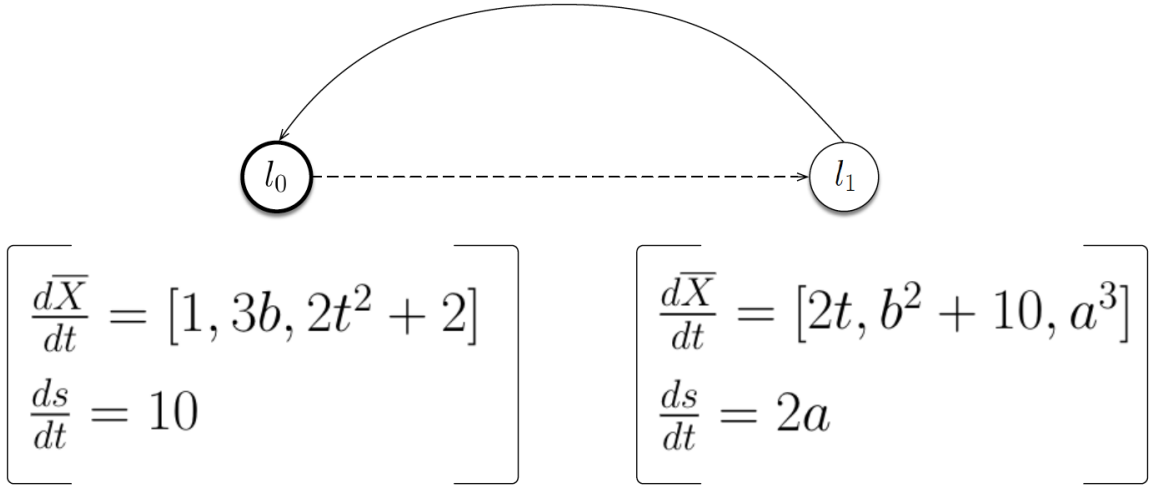


Figure 3.6

It is not necessary to explicitly list the continuous variables in X; the variables for which no dynamics are given are redundant when seeking a unique representation for the state-space of the system; omitting them does not reduce the expressibility of the language accepted by the automaton.

The guards on a transition are given as bracketed systems of inequalities, adjacent to the edge representing the transition. The bracketed system also contains the resets

and the event label associated with the transition:

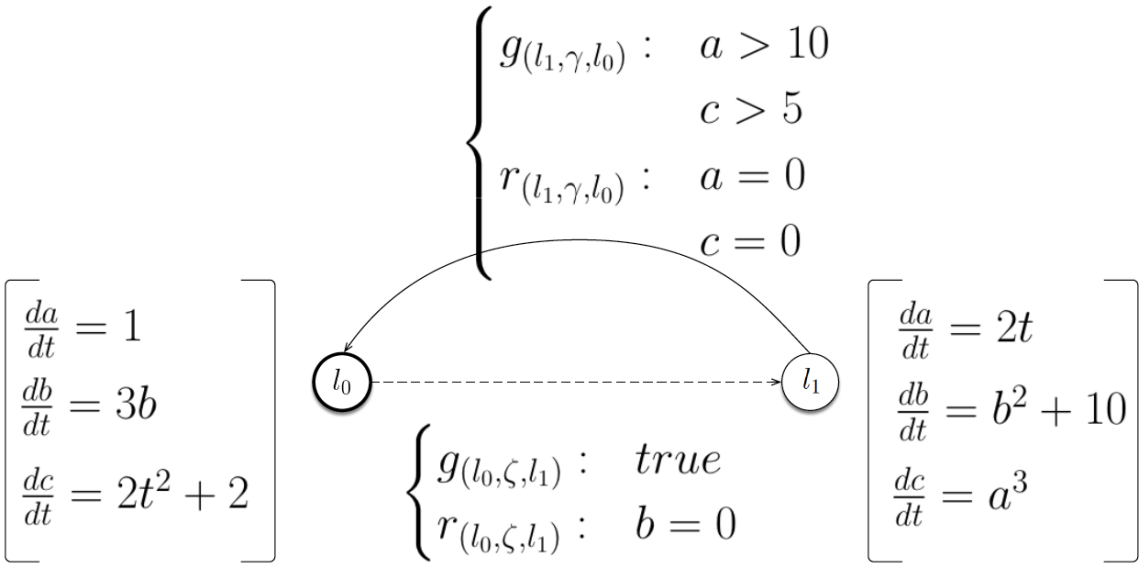


Figure 3.7

Many of the automata presented in this work have a finite, but indeterminate, number of locations. It is important, then, to be able to compactly express automata that would otherwise be very large in expanded graphical notation upon enumerating all locations $l_0, l_1, ..., l_n$.

The notation for doing so is as follows:

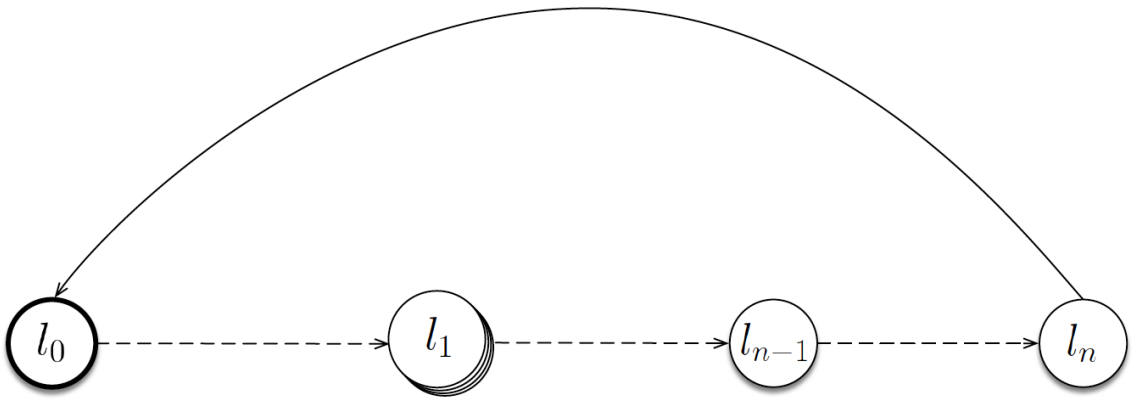


Figure 3.8

The locations l_3 up to l_{n-2} are "hidden" under location l_2. While concise, it is impossible to explicitly define transitions, guards, and resets between such locations. Hence, this representation should only be used when the properties of the transitions can be inferred from the transition immediately preceding (l_0 to l_1) and succeeding (l_{n-2} to l_{n-1}).

In the two automata that follow, the transitions, guards, resets, and dynamics on the obscured locations should be able to be reasonably inferred from the transitions and dynamics that are explicitly given:

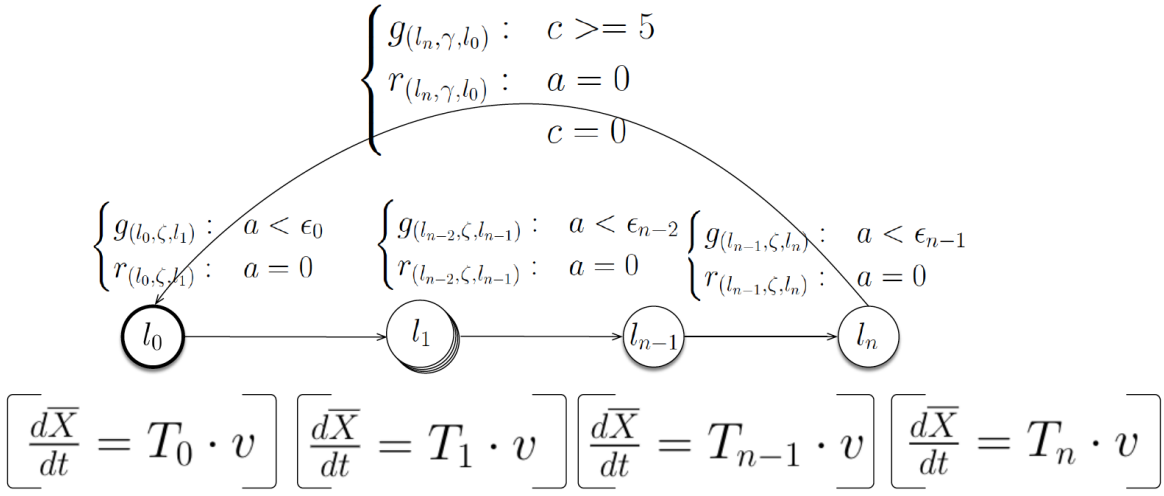


Figure 3.9

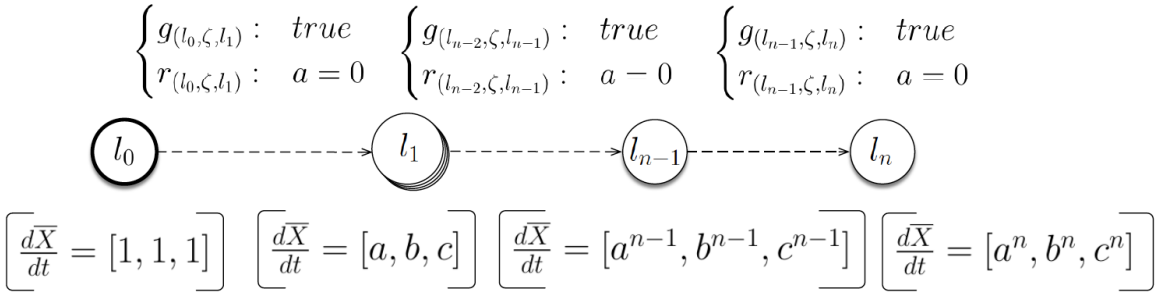


Figure 3.10

For visual clarity, some transitions from multiple locations to a singular location

may be merged, in the event that they have the same guards and resets:

Chapter 4

Literature Review

In this section, the existing work that applies hybrid systems (including but not limited to *hybrid automata*) to the task of modelling cellular metabolism is reviewed and a number of core conclusions are drawn. Many of these approaches incorporate discrete switching to model regulatory mechanisms, as a discrete state changed followed by shift in dynamics, triggered in response to threshold being crossed for some continuous variable, is a natural framework to represent a change in gene expression (and hence, active metabolic pathways) by means of cellular regulation. In fact, as the internal control mechanism of the cell, cellular regulation provides a natural area for the application of control theoretic techniques to better understand intracellular behavior.

In [Modeling Treatment and Drug Effects at The Molecular Level Using Hybrid System Theory], a gene regulation network is modelled using a system of piecewise linear/affine equations. Switching dynamics are provided by unit step functions embedded in the continuous equations, which in turn regulate the levels of gene expression in response to their activation/deactivation. The dynamics of each gene are enhanced with the addition with an external drug term, which represents the level at which the external substance increases/decreases expression of that gene. Simulations can then

be run, analyzing the change in dynamics in response to the addition of the drug term for *in silico* drug discovery. This model, however, does not directly consider metabolism and only includes regulatory mechanisms known at the time of formulating the model for some cell. Hence, it provides little insight into the nature of the metabolic shifts that are induced in response to a change in the extracellular medium.

In [Modeling and simulation of metabolic networks for estimation of biomass accumulation parameters], steady-state internal metabolism is coupled with external uptake/secretion dynamics and the behavior of the cell is formulated as a two-location state machine. The internal metabolism is defined by the steady-state stoichiometric equation $Sv = 0$ which is then solved with respect to an objective function. The uptakes/secretions are then determined by the internal flux values selected by the objective function and are bounded by kinetic constraints. However, instead of using the two-step iterative process to simulate metabolism as is done in dFBA, the internal and external dynamics are coupled into a differential-algebraic system of equations (DAE) which is directly solved for. This is a fairly generic approach that requires a limited amount of information about the organism to be known ahead of time and in theory could be applied to simulate a wide variety of organisms. However, solving the system is challenging and cannot be solved with standard solvers; the authors had to use a modified form that uses approximations of the discontinuous terms to solve it in MATLAB. Hence, this approach does not seem to scale well to arbitrary systems as there are no guarantees that the approximations that will need to be made to larger systems with a greater number of discontinuities will faithfully represent the behavior of the original system in question. Furthermore, the system does not provide a way to isolate the different regulatory states that may be induced by shifts in the metabolic network.

In [Formalizing metabolic-regulatory networks by hybrid automata], metabolic and regulatory dynamics are combined to build a hybrid automaton model. Each discrete

location is used to represent a specific regulatory mode, wherein the intra-cellular concentration dynamics are given by Michaelis-Mentens kinetic equations. Different regulatory modes represent changes in gene expression by "turning on/off" specific Michaelis-Menten terms in the concentration dynamics and the regulatory shifts are induced by the concentration of some metabolite crossing an established guard, triggering the change from one location to the next. This model is interesting as it uses the standard hybrid automata formalism to faithfully represent metabolic dynamics with a regulation component. Furthermore, the results were able to be duplicated by creating an identical hybrid automaton in the Ariadne library and running a similar set of simulations over it. These are an attractive set of propositions, but the weakness of this modelling approach comes from its inherent lack of generalizability. For a larger system, there is no efficient way to determine proper coefficients for every reaction to plug into the associated kinetic term. Furthermore, the metabolic pathways in the paper are not genome-scale, but rather made with a drastic set of simplifying assumptions to reduce the number of reactions (and hence, kinetic terms) that will need to have coefficients determined. This presents a trade-off between an accurate model that's impossible to practically assemble and an simplified one that may lose granularity, especially when considering more complex systems. Additionally, like other hybrid systems models that incorporate regulation, this approach assumes that regulatory modes are known before assembling the model and offers no ability to unravel their structure as emergent properties of the metabolic network.

In [Symbolic Systems Biology: Hybrid Modeling and Analysis of Biological Networks], the authors present the notion of "symbolic systems biology", a heuristic for representing and reasoning about metabolic pathways focusing on their qualitative, instead of quantitative, behavior. This is introduced because highly quantitative models are often difficult to appropriately determine in practice, as determining the "real"

kinetic constants for each reaction a daunting if not computationally intractable task. In face of this difficulty, many models in practice determine the kinetic constants by fitting the model as a whole to experimentally gathered data, using min-sum-of-squares or similar model fitting techniques. The quasi-experimentally determined models are often highly sensitive to variance and noise in the determined parameters and there is no inherent guarantee that they faithfully represent the characteristics of the underlying biological system. Hence, the authors determine that it is preferable to have an inherently meaningful model and strive to be qualitatively, instead of quantitatively, correct in terms of its implementation and interpretation.

They build two examples of this approach, the first of which represents human glucose metabolism with a piece-wise linear system and uses bounded nondeterminism to account for experimental under-determination. This model takes insulin levels as environmental inputs and uses compartmental representation for the different physiological components that use insulin levels as a trigger start a cascade of glucose uptake. The piece-wise linear system is an abstraction of an underlying nonlinear model that contains switching behavior using sigmoidal functions. The authors then perform a reachable set computation to determine bounds for the output of the first segment of the network, that responds to insulin. Using the stability theory of linear differential equations, eigenvalues for the system are computed which show that the dynamics of the second half of the network are stable under both the upper bound and lower computed in the latter segment. This network also provides an *in silico* model for testing insulin injections on a diabetic patient, which can be simulated by setting the biological insulin level to zero and computing the reachable set after supplemental insulin is added to the system.

, and such models are often highly sensitive to variations in

In [Hybrid Modeling and Simulation of Biomolecular Networks], the authors detail

a domain-specific modelling language that incorporates hybrid dynamics, nondeterminism, and stochastic variables to model a protein network that self-regulates through a closed feedback loop. Continuous dynamics are applied to model mRNA concentrations when they are above a determined level, wherein individual particle dynamics and collisions can be ignored in favor of treating the mRNA in the medium as a fully-diffused mixture. At lower levels, however, this continuous, deterministic approximation does not faithfully represent the discrete nature of the particles and the unpredictability of their collisions with one another. Hence, when the mRNA concentration drops below the aformentioned level, the continuous dynamics are substituted in favor of a stochastic model of the interaction behavior between protein and mRNA with updates to the discrete concentrations over a fixed time-step. Another example is also presented that models quorum sensing among a bacterial culture. Intra- and extra-cellular continuous dynamics are coupled with discrete switching behavior that is triggered in response to extracellular concentration of a signalling molecule. Hence, the microbes in the culture are able to collectively detect when they are in "quorum" by the extracellular concentrations which then governs their internal dynamics (they have three disjoint internal states).

Given the broad areas of application, much work has already been performed towards modeling the metabolism of microbial co-cultures using FBA and other constraint-based methods. However, there is a lack of literature pertaining to the use of hybrid systems to model microbial metabolism, much less the specific (and more challenging) case of microbial co-cultures. The majority of the existing work applying hybrid systems (hybrid automata in particular) to the modeling and analysis of microbial metabolism is limited in scope as well as potential application. This is because

1. Most of the approaches to model metabolism using hybrid systems involve directly modeling the detailed enzyme kinetics. For most organisms, the mech-

anisms governing each reaction are not known, and it is very unlikely that a complete characterization of all enzyme-catalyzed reactions will be found for any sufficiently large number of microbial organisms under consideration. Even for a single organism of interest, with hundreds to thousands of reactions, elucidating and quantifying the enzymatic mechanism for each reaction becomes infeasible and intractable.

2. Due to the latent difficulty of appropriately characterizing the mechanisms underlying each reaction, the network itself can be simplified on the order of phenotypically-distinct pathways. This renders a kinetic model where entire pathways, instead of reactions, are the basic unit for which kinetic equations are defined. While workable for organisms with well-understood metabolic pathways and regulatory mechanisms, the same problems as the reaction-centric approach are invoked upon generalizing to. Namely, the kinetic equations themselves

   Hence, simplifying assumptions about mechanisms are usually made which contradicts the purpose of a detailed kinetic model. Two key objections to this type of kinetic modeling are that the transferability of equation forms among organisms is unknown and the even the transferability of enzyme kinetic behavior between the test tube and the cellular interior is debatable.

3. The problem of modeling the detailed enzyme kinetics is further complicated when a co-culture is considered, where in addition to the reactions within each organism, the detailed dynamics of the metabolites in the medium also needed to be tracked.

In light of the above observations, it can be concluded that hybrid systems approaches to model microbial metabolism either require either a great deal of preexisting knowledge of reaction mechanisms, drastic simplifying assumptions, or both. Hence,

the use of hybrid systems approaches to develop models for metabolism has not seen much popularity. Since the lack of knowledge on enzyme kinetic mechanisms and related parameters can be bypassed using the constrained-based analysis methods that utilize reaction stoichiometries instead of enzyme kinetics, it is worthwhile to develop an approach that builds hybrid systems models for a microbe given only knowledge of the organism's stoichiometry. Knowledge of reaction mechanisms can be useful for bounding fluxes, but is not necessary.

There are a number of connections drawn in the literature between the behavior of regulatory networks and an engineered control system. Indeed, it is suggested that the only regulatory networks that should emerge in a natural biological system are those that are close to an optimal controller for maximizing some biological criterion. Hence, the expected behavior of some regulatory network should map elegantly onto the theory of optimal control, which, as a well-formulated theory, provides both descriptive ability for modelling a regulatory mechanism as well as predictive power for 1) forecasting how such mechanisms will behave in a number of environmental conditions 2) determining which regulatory mechanisms are realistic out of a number of potential candidates, given the assumption that they should be near-optimal controllers.

Another promising area that is underexplored in the existing literature is the application of stochastic hybrid systems to intra-cellular modelling (and more broadly, biological modelling in general). Similar to the benefits achieved by employing nondeterministic representation of parameters, stochastic variables are able to represent the noise inherent in biological systems

Advantages of our modelling approach:

1. Possesses the ability to construct rigorous metabolic models using only knowledge of stoichiometry

2. Is highly generalizable, uses the canonical formalism for hybrid automata such that it can be plugged into a number of standard solvers and does not necessitate building bespoke tools

3. Harnesses nondeterminism to allow exhaustive analysis of metabolic capacity

4. Using the theory of extreme pathways and eigenpathways, can easily be extended to incorporate regulation without loss of generality

5. Provides a natural formalism in which to apply control theoretic techniques to regulation, that also incorporates a metabolic component

6. Hence, this is a metabolic-regulatory model that can be dynamically constructed from minimal information about the organism at hand (outside of stoichiometric data, which is widely available) and can be integrated with a control-theoretic component to explore regulatory mechanisms, or left nondeterministic to exhaustively explore emergent metabolic capacity

7. Is highly visualizable, lends itself to visuals for better understanding of the model's capacity as well as identification of structural similarity with other models

Chapter 5

Metabolic Modelling with Hybrid Automata

5.0.1 Formulation of a Reference Model

The model organisms presented in Chiu et al.'s detection procedure for emergent metabolism [5] directly provide the stoichiometry, bounds, and dynamics for the reference microbes in this work. Chiu's work gives a) a highly granular description of the dFBA procedure used to detect emergent metabolites, including parameters of simulation, and b) a detailed analysis (with visualizations) of the expected qualitative behavior. Thus, Chiu's work serves as an an excellent launchpad for any further investigation of emergent metabolism on the cellular level – it gives input models with a set of simulation results to validate against a new approach.

The microorganism models, which shall be denoted Organism 1 and Organism 2, are defined as follows:

Organism 1: The internal stoichiometric matrix contains the reaction stoichiometries for intracellular metabolites, all of which exhibit steady-state behavior, whereas

the uptake and secretion stoichiometric matrix shows the reaction stoichiometries for external metabolites in the growth medium.

Internal Stoichiometric Matrix

$$\begin{bmatrix} - & R_1 & R_2 & R_3 & R_4 & J_1 & J_2 & J_3 & J_4 \\ \frac{dA}{dt} & -1 & 0 & 0 & 0 & -1 & 0 & 0 & 0 \\ \frac{dD}{dt} & 1 & -3 & 0 & -1 & 0 & 0 & 0 & 0 \\ \frac{dZ}{dt} & 0 & 1 & -10 & 1 & 0 & 0 & 0 & 0 \\ \frac{dW}{dt} & 0 & 2 & 0 & 0 & 0 & 0 & 0 & -1 \\ \frac{dC}{dt} & 0 & 0 & 0 & -1 & 0 & -1 & 0 & 0 \\ \frac{dY}{dt} & 0 & 0 & 0 & 1 & 0 & 0 & -1 & 0 \end{bmatrix}$$

Uptake & Secretion Stoichiometric Matrix:

$$\begin{bmatrix} - & R_1 & R_2 & R_3 & R_4 & J_1 & J_2 & J_3 & J_4 \\ \frac{dA_{env}}{dt} & 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ \frac{dC_{env}}{dt} & 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ \frac{dW_{env}}{dt} & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ \frac{dY_{env}}{dt} & 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

Internal Concentrations	**External Concentrations:**
$dA/dt = -R_1 - J_1$	$dA_{env}/dt = J_1 * Bio$
$dD/dt = R_1 - 3R_2 - R_4$	$dC_{env}/dt = J_2 * Bio$
$dZ/dt = R_2 - 10R_3 + R_4$	$dW_{env}/dt = J_4 * Bio$
$dW/dt = 2R_2 - J_4$	$dY_{env}/dt = J_3 * Bio$
$dC/dt = -R_4 - J_2$	
$dY/dt = R_4 - J_3$	

Organism 2:

Internal Stoichiometric Matrix

$$\begin{bmatrix} - & R_5 & R_6 & J_5 & J_6 & J_7 \\ \frac{dB}{dt} & -1 & 0 & 0 & -1 & 0 \\ \frac{dX}{dt} & 1 & -50 & 0 & 0 & 0 \\ \frac{dC}{dt} & 1 & 0 & 0 & 0 & -1 \\ \frac{dA}{dt} & -1 & 0 & -1 & 0 & 0 \end{bmatrix}$$

Uptake & Secretion Stoichiometric Matrix:

$$\begin{bmatrix} - & R_5 & R_6 & J_5 & J_6 & J_7 \\ \frac{dB_{env}}{dt} & 0 & 0 & 0 & 1 & 0 \\ \frac{dC_{env}}{dt} & 0 & 0 & 0 & 0 & 1 \\ \frac{dA_{env}}{dt} & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

Internal Concentrations	**External Concentrations:**
$dB/dt = -R_5 - J_6$	$dB_{env}/dt = J_5 * Bio$
$dX/dt = R_5 - 50R_6$	$dC_{env}/dt = J_6 * Bio$
$dC/dt = R_5 - J_7$	$dA_{env}/dt = J_7 * Bio$
$dA/dt = -R_5 - J_5$	

Figure 1 illustrates the interaction among the organisms.

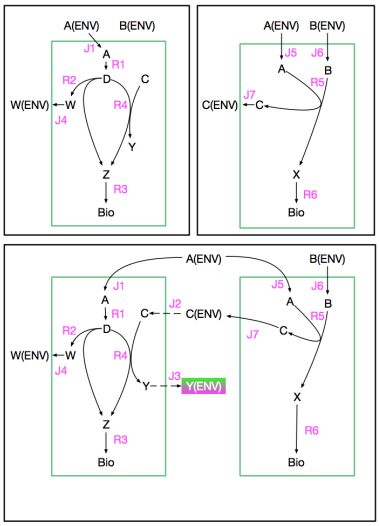


Figure 5.1: Visualization of the co-culture formed by Organism 1 and Organism 2. Note the interaction between them by means of metabolite C, which Organism 1 processes to yield metabolites Y and Z.

Note that the nutrient uptake and metabolite secretion rates are given as functions of the biomass of the organism as well as the flux values. This is done to enforce the physically realistic condition that a larger organism possesses greater uptake potential while simultaneously requiring more of the necessary nutrients to survive and grow.

While the biomass in our model, denoted as Bio, could be considered an internal metabolite concentration (biomass is not shared between organisms in the extracellular medium), $dBio/dt$ is not included in the (internal) stoichiometric matrix S because the steady-state condition $S \cdot v = 0$ does not apply to the biomass. Moreover, with increase in biomass, there eventually occurs creation of new microbial cells which we are not explicitly considering in our model. Thus increase in biomass is intimately tied to the concentration of metabolites in the extracellular medium. Hence, biomass itself is treated as an external metabolite. Indeed, it is desirable for biomass to increase, i.e. for growth to occur as much as possible within a time-step. The unique dynamics of Bio are handled as a special case that will be further elaborated when discussing

the modelling approach in detail. As shown in Fig 5.1, for all intents and purposes, in Organism 1, consider that R_3 alone contributes to the biomass and that $dBio/dt = R_3$. Likewise, in Organism 2, $dBio/dt = R_6$.

The bounds on the transport reactions are taken as the minimum of the Michaelis-Menten equations and maximum consumption rate per cell per time-step[5]. The Michaelis-Menten equations give the reaction rate for single-substrate reactions with the intermediate enzyme complex concentration fixed for short time-scales (quasi-steady-state assumption). Since flux-balance analysis is solely concerned with the fluxes that produce metabolic equilibrium (all uptakes and all secretions sum to zero), the Michaelis-Menten equations provide a limit for the rate of conversion at any given substrate concentration. The maximum consumption rate per cell per time-step considers the case wherein the environmental concentration of a nutrient is fully consumed by the organism(s) over the fixed time-step, Δt. Hence, the rate of uptake should be scaled proportionally to both the total biomass (each unit of biomass is allocated an equal amount of nutrient) and the size of time-step (to ensure a constant uptake rate throughout the entire interval).

Thus, for any uptake flux J_x associated with metabolite C_k, $1 \leq k \leq M$, the lower bound at time t is given as:

$$-min\{\frac{V_m * C_k}{K_m + C_k}, \frac{C_k}{Bio * \Delta t}\} \leq J_x \leq \infty$$

A lower bound instead of an upper bound is used to limit metabolite uptake from the growth medium, as uptake reactions are conventionally assumed to have a stoichiometric coefficient of -1. The internal fluxes themselves remain unbounded.

5.1 Flux-Based Modelling Approaches

5.1.1 Decomposing Steady-State Metabolism

Dynamic FBA (dFBA) simulates the metabolism of an organism by solving for the steady-state fluxes that maximizes growth as the objective function, setting the resulting fluxes as constant for a sufficiently short time interval. The simulated growth flux is used to update the concentrations of biomass and metabolites in the extracellular medium at the end of the time step. The process is then iterated for the succeeding time-steps.

While the dFBA algorithm clearly defines the trajectory of a continuous dynamical system, representing the underlying biological system as a hybrid automaton in its most generic form proves challenging. This is due to a) the inherent nondeterminism of the process, b) the difficulty of identifying the individual discrete states, and c) the algebraic constraints ($S \cdot v = 0$) on the flux values.

To elucidate further: the metabolite concentrations (internal or external) are governed by dynamics that depend on unknown flux values. Hybrid automata are able to represent systems with parameterized dynamics. Many model checkers for hybrid automata (among them, Flow*, HyPro) support verification on systems with parameterized dynamics – given that the unknown parameters are at least bounded from one side. However, adding algebraic constraints to the parameters (like the steady-state requirement $S \cdot v = 0$, where v is the flux vector) turns the dynamics of each state into a differential-algebraic system of equations (DAE). Differential-algebraic hybrid automata are significantly harder to reason about than regular hybrid automata, and while they have been used successfully to analyze biological systems, no general purpose model checker exists that can handle all biologically relevant cases where they occur. Thus, a different approach is desired – one that maintains the generality of

the biological system under consideration while falling nicely within standard hybrid automata representations.

While the final modelling approach fully generalizes any trajectory that can be computed by dFBA, handling the specific, deterministic case is first necessary to build the structural induction argument behind the general hybrid automaton. A hybrid automaton should be introduced that computes one trajectory for the microbe in question, as opposed to the desired approach, which uses nondeterminism to compute any biologically valid trajectory.

The dFBA formulation for a specific organism induces a dynamical system given by the tuple $\mathbb{D} = (\overline{C}_{int}, \overline{C}_{ext}, S, T, v_{max}, v_{min}, \overline{C}_0, \Delta t, t_{max})$, where $\overline{C}_{int}$ and $\overline{C}_{ext}$ are vectors of the internal and external metabolite concentrations, S and T are matrices corresponding to the internal stoichiometry and uptake / secretion stoichiometry, v_{max} and v_{min} are the upper and lower bounds (respectively) for the vector of fluxes, $\overline{C}_0$ is the initial value vector for the metabolite concentrations ($[\overline{C}_{int}, \overline{C}_{ext}] = \overline{C}_0$ at $t = 0$), and t_{max} is the end time of the trajectory. For a specific microbial model within the dFBA formulation, the stochiometric matrices S and T and the bounds for each flux are known and not simulation-dependent. The size of the timestep, Δt, is determined before calculating any trajectory and is fixed for the duration of the simulation. Thus, it can be treated as a constant with respect to the dFBA in question. The number of time steps is given by $n = \frac{t_{max}}{\Delta t}$. Within each time-step, $\overline{C}_{int}$ and $\overline{C}_{ext}$ define the state of the system while the flux values parameterize the concentration dynamics ($\frac{d\overline{C}_{int}}{dt}$, $\frac{d\overline{C}_{ext}}{dt}$). Thus, given a set of initial concentrations, the selection of the flux vector v in each time-step is the only nondeterministic aspect of the trajectory.

It is to be noted that the objective function $G(v)$ that is usually given when describing dFBA is notably absent – this is because the objective function often results in a nondeterministic selection of v from several satisfactory options, so $G(v)$ itself

is entirely arbitrary when precisely defining both the general dynamical system as well as a specific trajectory of the system. Trajectories are instead given directly by $\mathbb{T} = (\mathbb{D}, \overline{V} = [v_1, v_2, v_3..., v_n])$, where n is the number of time steps and $\overline{V}$ is a sequence of flux vectors such that the flux vector $[\overline{V}]_k = v_k$ governs the concentration dynamics for the timestep ending at $k * \Delta t$. As shorthand, a specific trajectory of any $\mathbb{D}$ is denoted as $\mathbb{D}(\overline{V})$. Using the previous formalism for dFBA, a hybrid automaton that exhibits the trajectory $\mathbb{D}(\overline{V})$ can be built by using the procedure in Algorithm 1. Consider that for $\mathbb{D}(\overline{V})$, the continuous trajectory is divided into equal discrete intervals. The first interval of size Δt has the concentration dynamics $d\overline{C}_{int}/dt = S * v$ and $d\overline{C}_{ext}/dt = T * v * Bio$ and parameterized by $[\overline{V}]_1 = v_1$. After time Δt passes, a discrete "switch" occurs and the dynamics are then parameterized by $[\overline{V}]_2 = v_2$. This switching, in practice, is forced by a "clock" variable, τ, where $d\tau/dt = 1$ and occurs upon $\tau > \Delta t$. τ is then reset to 0. As this switching behavior only changes the parameterization of the dynamics, the concentrations themselves can be updated continuously throughout the process. Possessing a continuous state space with dynamics that "switch" at fixed intervals, it should be obvious that $\mathbb{D}(\overline{V})$ can be easily represented as hybrid automata. As the dynamics of Bio are not given in $\mathbb{D}$, consider Bio to be constant and equal to 1.

It is known that any valid v exists within the flux-cone induced by stoichiometry S and the bounds v_{max} and v_{min}. By definition, v satisfies the equation $S \cdot v = 0$ which guarantees internal steady-state. Since the dynamics of the internal concentrations are given as $d\overline{C}_{int}/dt = S \cdot v$, $d\overline{C}_{int}/dt = 0$ for all values of v. Hence, $d\overline{C}_{int}/dt$ and $\overline{C}_{int}$ are not necessary to uniquely describe the state space of the system and can be discarded in the final model, leaving $X = \overline{C}_{ext} \cup \{\tau\}$ as the set of continuous variables. Accordingly, $\overline{C}_0$ should be restricted to only variables from X as well.

Algorithm 1: Transformation of dFBA trajectory $\mathbb{D}(\overline{V})$ to hybrid automaton $\mathbb{H}(\mathbb{D}, \overline{V})$

Input: The trajectory $\mathbb{T} = (\mathbb{D}, \overline{V} = [v_1, v_2, v_3..., v_n]) = \mathbb{D}(\overline{V})$;

Result: Hybrid automaton $\mathbb{H}(\mathbb{D}, \overline{V})$ that follows trajectory $\mathbb{D}(\overline{V})$;

Initialize a hybrid automaton $\mathbb{H}$ where:

    Continuous variables $X = \overline{C}_{ext} \cup \{\tau\}$;

    Discrete locations $L = \{l_1, l_2, ..., l_n\}$;

    Initial condition $I = (l_1, [\overline{C}_0, 0])$;

    Assign the dynamics of l_1 as:

        $\frac{d\overline{C}_{ext}}{dt} = T \cdot v_1 * Bio$;

        $\frac{d\tau}{dt} = 1$;

for $i = 2; i <= n$ **do**

    Assign the dynamics of l_i as follows:

        $\frac{d\overline{C}_{ext}}{dt} = T \cdot v_i * Bio$;

        $\frac{d\tau}{dt} = 1$;

    Add a forced transition from l_{i-1} to l_i with guard condition $\tau > \Delta t$ and reset $\tau = 0$;

end

Return $\mathbb{H}$;

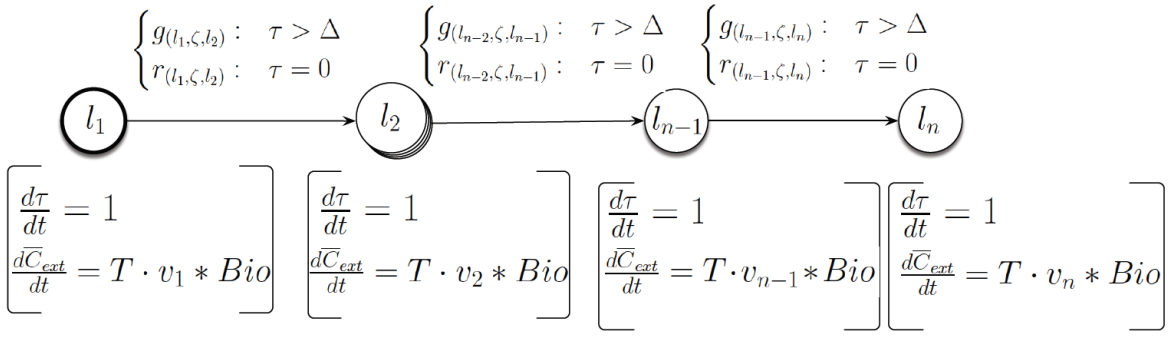


Figure 5.2: Visualization of $\mathbb{H}(\mathbb{D}, \overline{V})$ for an arbitrary $\mathbb{D}(\overline{V})$

The output automaton can be denoted by $\mathbb{H}(\mathbb{D}(\overline{V})) = \mathbb{H}(\mathbb{D}, \overline{V})$. During the construction process, the introduction of the clock variable, τ is the only explicit addition needed to represent $\mathbb{D}(\overline{V})$ as a hybrid automaton as shown in Fig 5.2. All other parameters come directly from $\mathbb{D}(\overline{V})$ itself.

While the construction $\mathbb{H}(\mathbb{D}, \overline{V})$ is a necessary step in designing a hybrid automaton that encapsulates $\mathbb{D}$, it is limited by its explicit dependence on $\overline{V}$. $\mathbb{D}$ can take any number of trajectories where $\overline{V}$ satisfies the stoichiometry and bounds. An automaton, $\mathbb{H}(\mathbb{D})$, is sought that can do the same that does not explicitly require $\overline{V}$ as a parameter for construction, initialization, and simulation. In particular, since $\overline{V}$ is chosen in a nondeterministic fashion, $\mathbb{H}(\mathbb{D})$ should be able to nondeterministically take any trajectory $\mathbb{D}(\overline{V})$.

To start, the procedure for constructing $\mathbb{H}(\mathbb{D})$ should be limited to the simplest case, $\overline{V} = [\overline{V}]_1 = v_1$, a singular constant flux, v_1 non-specified. Nondeterministic dynamics for an unspecified v_1 can be represented within the automaton by deconstructing the flux cone into its basic components. To recall, the flux cone is the convex polytope defined by the bounds and stoichiometry that contains all valid fluxes of the organism. Formally, the flux cone $\mathbb{F} \subset \mathbb{R}^N$, where $\mathbb{F} = \{v \mid S \cdot v = 0 \wedge v_{min} \leq v \leq v_{max}\}$. As the bounds themselves can be time-dependent, $\mathbb{F}$ is a differential inclusion. Denote the flux cone at time t as $\mathbb{F}_t$. By ignoring the flux bounds and focusing only on the subspace defined by the stoichiometry, one receives $\mathbb{F}_\emptyset = \{v \mid S \cdot v = 0\}$, where $\forall t \geq 0, \mathbb{F}_t \subset \mathbb{F}_\emptyset \subset \mathbb{R}^N$. $\forall v, v \in \mathbb{F}_t \Rightarrow v \in \mathbb{F}_\emptyset$. The general solution of $S \cdot v = 0$ gives the linearly independent vectors $B = \{b_1, ..., b_n\}$, where B form the bases of $F_\emptyset$. Thus, $\forall v \in \mathbb{F}_t$, $v = \sum_{i=1}^{n} b_i * k_i, k_i \in R$.

Using the previous representation of v, the automaton $\mathbb{H}(\mathbb{D})$ can be built for any v_1. Specifically, the equation $S \cdot v = 0$ is solved and the bases $B = \{b_1, ..., b_n\}$ are found. This procedure can be done using MATLAB or any standard numerical soft-

ware package. Add n discrete locations to the automaton, $L = \{l_1, l_2, ..., l_n\}$, each location endowed with the same continuous variables and parameterized dynamics as the locations in $\mathbb{H}(\mathbb{D}, \overline{V})$. Each basis should then be "assigned" to a corresponding location in the automaton, $b_1 \Rightarrow l_1, ..., b_n \Rightarrow l_n$, such that the basis vector parameterizes the dynamics of the location as shown in Fig 5.3. Add nondeterministic (unforced) transitions with guard condition "true" from each location to the following location, where "following" is defined as the location l_{j+1} for a location $l_j, j < n$. If $l_j = l_n$, there is no following location and thus no outgoing transition is added. In the same way as $\mathbb{H}(\mathbb{D}, \overline{V})$, the dynamics of internal metabolites will always be steady-state, so $\overline{C}_{int}$ can be discarded from the final model, leaving $X = \overline{C}_{ext} \cup \tau$ as the set of continuous variables. Accordingly, $\overline{C}_0$ should be restricted to variables from X as well. S is initially needed to find the bases that describe the flux cone, but can be discarded in the final model as it only defines internal stoichiometry and dynamics.

Combining the parameterized dynamics with unforced transitions allow the dynamics of $\mathbb{H}(\mathbb{D}, v_1)$ to be "decomposed" generically such that the trajectory associated with any v_1 can be taken. However, the primary automaton of interest is a generic representation of $\mathbb{H}(\mathbb{D}, \overline{V} = [v_1, v_2, ...,])$, $\overline{V}$ unspecified. This is accomplished by adding a nondeterministic (unforced) transition from l_n back to l_1, with guard condition "true". Hence, the transitions of the automaton form a cycle, where each "forward" sequence of transitions (l_1 to l_n) possesses identical dynamic behavior to $\mathbb{H}(\mathbb{D}, v_i)$. The transition from l_n back to l_1 equates to the change in parameterization from some v_i to v_{i+1} within $\mathbb{H}(\mathbb{D}, \overline{V})$. Hence, by "unwrapping" $\mathbb{H}(\mathbb{D}, \overline{V} = [v_1, v_2, ...,])$ into a sequence of automata [$\mathbb{H}(\mathbb{D}, v_1)$, $\mathbb{H}(\mathbb{D}, v_2)$, ...] to be simulated in succession, then by "decomposing" the dynamics of each $\mathbb{H}(\mathbb{D}, v_i)$ into computationally equivalent trajectories composed of "building block" dynamics, a generic automata $\mathbb{H}(\mathbb{D})$ for $\mathbb{H}(\mathbb{D}, \overline{V})$ is built.

For the automata $H_1 = \mathbb{H}(\mathbb{D})$ and $H_2 = \mathbb{H}(\mathbb{D}, \overline{V} = [v_1, v_2, ..., v_n])$, it must be shown

that any trajectory achievable by H_2 is also achievable in H_1. For the purposes of simulation, the changes in the clock variable τ, $d\tau/dt = 1$, should not be considered part of the "trajectory." Indeed, τ is only necessary to force transitions within H_2. Thus, any trajectory $\mathbb{D}(\overline{V})$ taken by H_2 that results in a change in concentration of a metabolite must also produce that same change in concentration when $\mathbb{D}(\overline{V})$ is taken by H_1, for all metabolites.

In location l_1 of H_2, let the external metabolite A with initial concentration A_0 be governed by the dynamics $\frac{dA}{dt} = F$, where F is a linear combination of the transport fluxes $J_1, ..., J_h$. $\frac{dA}{dt}$ is specifically given by $[T \cdot v_1]_s$.

It is known that $\mathbb{F}_t$ defines the flux cone at time t, $\mathbb{F}_t \subset F_\emptyset$, and $v_1 \in \mathbb{F}_t$. The bases $\{b_1, b_2, ..., b_n\}$ span the null space of S and hence span $F_\emptyset$. Since $v_1 \in F_\emptyset$, $v_1 = \sum_{k=1}^{n} b_k * m_k, m_k \in R$.

It is also known that v_1 has the structure $[R_1, ..., J_1, ..., J_h]$. Since $v_1 = \sum_{k=1}^{n} b_k * m_k$, for some $x, 1 \leq x \leq h$, $[v_1]_i = J_x = [\sum_{k=1}^{n} b_k * m_k]_i = \sum_{k=1}^{n} [b_k]_i * m_k$. As $\frac{dA}{dt} = F = [T \cdot v_1]_s = \sum_{j=1}^{h} J_j * g_j$, each J_j itself can be represented as a linear combination of bases that are then sub-indexed to the element corresponding to the J_j flux. This is accomplished by transforming $[b_k]_i$ to an indexing of j. Specifically, $i = |v_1| - h + j$. Putting this together, $\frac{dA}{dt} = \sum_{j=1}^{h} J_j * g_j = \sum_{j=1}^{h} [\sum_{k=1}^{n} [b_k]_{\{|v_1|-h+j\}} * m_k] * g_j$. This rather un-intuitive expression can be simplified by making two observations: 1) the m_k elements are the same for each iteration of the inner loop because v_1 is fixed, and 2) the indexed bases are the only terms that depend on k and j, so the order of the loops can be reversed. This gives $\frac{dA}{dt} = \sum_{k=1}^{n} [\sum_{j=1}^{h} [b_k]_{\{|v_1|-h+j\}} * g_j] * m_k$. This can also be represented as $\frac{dA}{dt} = [T \cdot \{\sum_{k=1}^{n} b_k * m_k\}]_s$. As the initial condition of A is A_0 and the dynamics $\frac{dA}{dt} = F$ are constant within l_1, the concentration of A upon leaving l_1 at time Δt (as forced by the transition from l_1 to l_2) is computed as $\int_{t=0}^{\Delta t} F dt = A_0 + \Delta t * F$.

In location l_1 of H_1, the initial condition of A is also A_0 but the dynamics are given by $\frac{dA}{dt} = [T \cdot b_1]_s$. Using the same steps as was done for l_1 of H_2, $[T \cdot b_1]_s = \sum_{j=1}^{h} [b_1]_{\{|v_1|-h+j\}} * g_j$. Thus, upon leaving l_1 at some arbitrary time (denoted t_1), the value of A is given with $\int_{t=0}^{t_1} \{\sum_{j=1}^{h} [b_1]_{\{|v_1|-h+j\}} * g_j\} dt = A_0 + t_1 * \sum_{j=1}^{h} [b_1]_{\{|v_1|-h+j\}} * g_j$. For shorthand, denote the previous $A_0 + t_1 * \sum_{j=1}^{h} [b_1]_{\{|v_1|-h+j\}} * g_j$ as A_1. A_1 is the initial concentration of A upon entering the location l_2 of H_1. Define t_2 as the elapsed time in the location l_2, or the time of transition $l_2 \Rightarrow l_3$ minus t_1. The dynamics of A within l_2 are given as $\frac{dA}{dt} = [T \cdot b_2]_s$. The value of A upon leaving l_2 is hence $\int_{t=0}^{t_2} \{\sum_{j=1}^{h} [b_2]_{\{|v_1|-h+j\}} * g_j\} dt = A_1 + t_2 * \sum_{j=1}^{h} [b_2]_{\{|v_1|-h+j\}} * g_j = A_0 + t_1 * \sum_{j=1}^{h} [b_1]_{\{|v_1|-h+j\}} * g_j + t_2 * \sum_{j=1}^{h} [b_2]_{\{|v_1|-h+j\}} * g_j$. Following the same steps, the concentration of A upon leaving the location l_n (completing a forward "run" of the automaton) is given as $A_0 + t_1 * \sum_{j=1}^{h} [b_1]_{\{|v_1|-h+j\}} * g_j + t_2 * \sum_{j=1}^{h} [b_2]_{\{|v_1|-h+j\}} * g_j + ... + t_n * \sum_{j=1}^{h} [b_n]_{\{|v_1|-h+j\}} * g_j = \sum_{k=1}^{n} [\sum_{j=1}^{h} [b_k]_{\{|v_1|-h+j\}} * g_j] * t_k$.

From the previous, it becomes apparent that any change in A produced in location l_1 from H_2 can be simulated in a forward "run" of H_1:

$\int_{t=0}^{\Delta t} F dt = A_0 + \Delta t * F = A_0 + \Delta t * \sum_{k=1}^{n} [\sum_{j=1}^{h} [b_k]_{\{|v_1|-h+j\}} * g_j] * m_k = A_0 + \Delta t * m_1 * \sum_{j=1}^{h} [b_1]_{\{|v_1|-h+j\}} * g_j + \Delta t * m_2 * \sum_{j=1}^{h} [b_2]_{\{|v_1|-h+j\}} * g_j + ... + \Delta t * m_n * \sum_{j=1}^{h} [b_n]_{\{|v_1|-h+j\}} * g_j = A_0 + t_1 * \sum_{j=1}^{h} [b_1]_{\{|v_1|-h+j\}} * g_j + t_2 * \sum_{j=1}^{h} [b_2]_{\{|v_1|-h+j\}} * g_j + ... + t_n * \sum_{j=1}^{h} [b_n]_{\{|v_1|-h+j\}} * g_j$

This equivalence establishes that, to simulate l_1 of H_2 using H_1, the values $[t_1, t_2, ..., t_n] = [\Delta t * m_1, \Delta t * m_2, ..., \Delta t * m_n]$ must be assigned. Hence, the total elapsed time needed to simulate the change in concentration is also found, where Δt in H_2 becomes $\Delta t * \sum_{k=1}^{n} m_k$ in H_1. After leaving l_n in H_1, l_1 is resumed and the process to simulate the dynamics of l_2 from H_2 can commence. l_2 is simulated by assigning $[t_1, t_2, ..., t_n] = [\Delta t * m'_1, \Delta t * m'_2, ..., \Delta t * m'_n]$, where $v_2 = \sum_{k=1}^{n} b_k * m'_k$. This process is iterated

to simulate the dynamics of $l_1, ..., l_{\frac{t_{max}}{\Delta t}}$ from H_2, where the original time $\Delta t * n$ needed to compute the trajectory $\mathbb{D}(\overline{V})$ directly through the dFBA framework becomes $\Delta t * \sum_{l=1}^{\frac{t_{max}}{\Delta t}} \sum_{k=1}^{n} m_k'^{(l-1)}$.

The previous procedure can be applied component-wise to every element of $\frac{d\overline{C}_{ext}}{dt}$, such that the result generalizes fully to all external concentration dynamics.

Algorithm 2: Transformation of dynamical system $\mathbb{D}$ to hybrid automaton $\mathbb{H}(\mathbb{D})$

Input: A dFBA procedure formulated as the dynamical system $\mathbb{D} = (S, T, v_{max}, v_{min}, \overline{C}_0, \Delta t, t_{max})$

Result: Hybrid automaton $\mathbb{H}(\mathbb{D})$

Solve $S \cdot v = 0$ to find the bases $B = \{b_1, ..., b_n\}$;

Initialize a hybrid automaton $\mathbb{H}$ where:

    Continuous variables $X = \overline{C}_{ext} \cup \{\tau\}$;

    Discrete locations $L = \{l_1, l_2, ..., l_n\}$;

    Initial condition $I = (l_1, [\overline{C}_0, 0])$;

for $i = 1; i <= |B|$ **do**

    Create location l_i in $\mathbb{H}$;

    Assign the dynamics of l_i as follows:

        $\frac{d\overline{C}_{ext}}{dt} = T \cdot b_i * Bio$;

        $\frac{d\tau}{dt} = 1$;

    Add an unforced transition from location l_{i-1} to location l_i with guard condition "true" and label η;

end

Add an unforced transition from location l_n to location l_1 with guard condition "true";

Return $\mathbb{H}$;

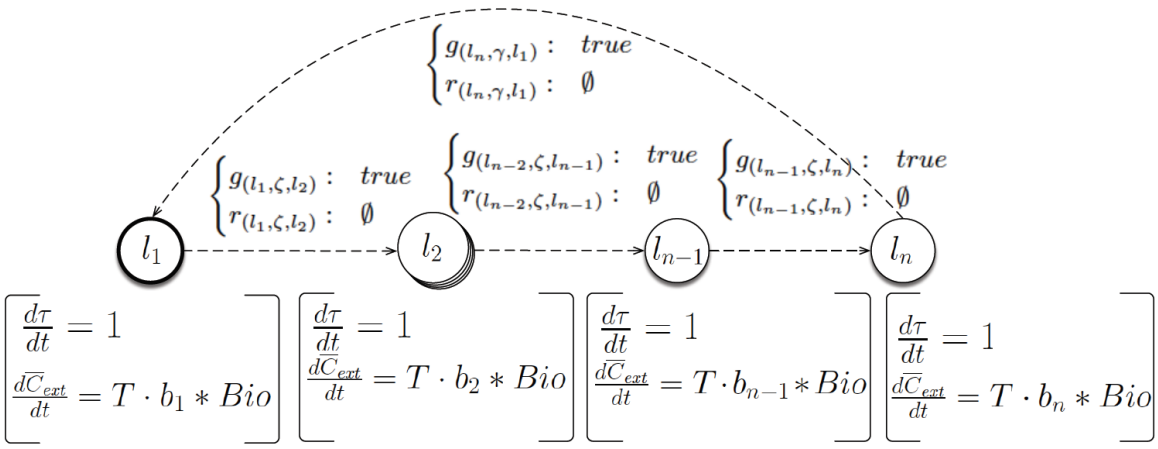


Figure 5.3: Visualization of $\mathbb{H}(\mathbb{D})$ for an arbitrary $\mathbb{D}$

5.1.2 Representing Biological Growth

While not technically a metabolite, the measure of biomass as a continuous variable Bio is useful for many dFBA formulations as biomass is often used in assigning bounds and defining the objective function. The dynamics of biomass are usually formalized as monotonically increasing and a function of one or more "internal" fluxes. While the dynamics could be given in the form $\frac{dBio}{dt} = v_i \cdot p$, $p \in R^{|v_i|}$, Chiu's model does not directly define $\frac{dBio}{dt}$. Instead, Bio is updated at the end of every time-step Δt as an exponential function of a singular flux value (R_3 for Organism 1).

This poses the question, then, of how to "find" the selected v_i (and by definition, R_3 or any other flux) upon the loop from l_n back to l_1 in H_1.

It is known that $v_i = \sum_{k=1}^{n} b_k * m_k, m_k \in R$. As the bases are fixed within H_1, $m_1, ..., m_n$ are the only degrees of freedom to determine the value of v_i. Furthermore, for each location $l_1, ..., l_n$ and the associated transition times $t_1, ..., t_n$, $[t_1, t_2, ..., t_n] = [\Delta t * m_1, \Delta t * m_2, ..., \Delta t * m_n]$.

So, $v_i * \Delta t = \sum_{k=1}^{n} b_k * m_k * \Delta t = \sum_{k=1}^{n} b_k * t_k$. As $\int_{t=0}^{t_k} b_k dt = b_k * t_k$, a vector of new variables with dimension $|v_i|$ should be added to X (or more properly, $\overline{X}$, as X itself

is not vectorized), termed $\overline{Q}$, where, in each location l_k, $\frac{d\overline{Q}}{dt} = b_k$, and $\overline{Q} = 0$ at t_0 and upon the transition $l_n \Rightarrow l_1$. Before resetting from $l_n \Rightarrow l_1$, the sequence of transitions $l_1 \Rightarrow l_2, ..., l_n \Rightarrow l_1$ produces $\overline{Q} = \int_{t=0}^{t_1} b_1 dt + ... + \int_{t=0}^{t_n} b_n dt = \sum_{k=1}^{n} b_k * t_k = v_i * \Delta t$ and thus realizes the vector "taken" in the forward "run" of the automaton. Furthermore, upon the transition $l_n \Rightarrow l_1$, Bio should be updated according to Chiu's model, as $Bio = Bio * e^{\Delta t * Bio_{agg}}$, where Bio_{agg} is the sum of the individual fluxes that contribute to biomass accumulation. Since $\overline{Q} = v_i * \Delta t$ already includes the Δt component, summing the necessary elements from $\overline{Q}$ is all that is required to produce $\Delta t * Bio_{agg}$. Create the vector Bio_{sel} (where each element is either 0 or 1) such that $Bio_{sel} \cdot \overline{Q} = \Delta t * Bio_{agg}$. The reset can be reformulated as $Bio = Bio * e^{Bio_{sel} \cdot \overline{Q}}$. As Bio is only updated upon the transition from $l_n \Rightarrow l_1$, Bio becomes a step function when plotted with respect to t. Furthermore, because Bio is constant within any forward "run", the dynamics of the internal concentrations can be defined as $\frac{d\overline{C}_{ext}}{dt} = T \cdot b_i * Bio$, $Bio \neq 1$ while respecting the previously established equivalence between H_1 and H_2.

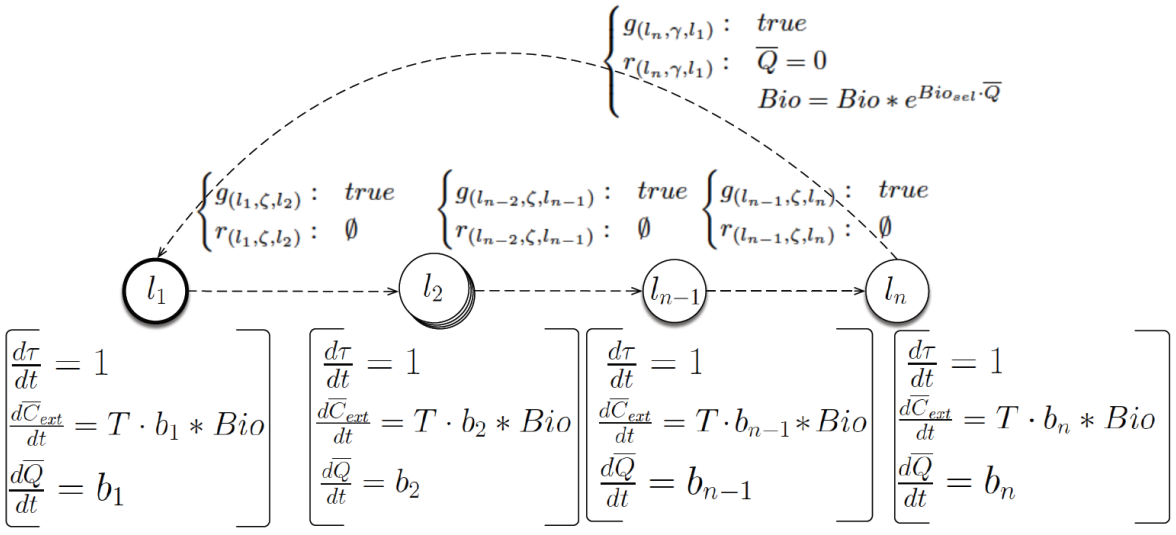


Figure 5.4: Output of Algorithm 2 with Biomass Accumulation

5.1.3 Incorporating Flux Bounds with Decomposed Dynamics

While the previous sections have established that, given a dFBA formulation $\mathbb{D}$, an automaton can be constructed to compute any arbitrary trajectory $\mathbb{D}(\overline{V})$, $\overline{V} = [v_1, v_2, ...]$ unspecified, an additional construction is necessary to fully represent the power of dFBA. Namely, $v_1, v_2, ...$ as previously defined are solutions to $S \cdot v = 0$ without considering the bounds $v_{min} \leq v \leq v_{max}$. Because the hybrid automata representation of v was derived from $span\{b_1, b_2, b_3, ..., b_n\} = \mathbb{F}_\emptyset$, $\exists v \in F_\emptyset \Rightarrow v \notin F_t$. Hence, the reachable set of $\mathbb{H}(\mathbb{D})$ is an overapproximation of the desired reachable set. The modified version of $\mathbb{H}(\mathbb{D})$ which respects flux bounds shall be labeled $\mathbb{H}(\mathbb{D}, v_{max}, v_{min}) = \mathbb{H}(\mathbb{D}, \mathbb{B})$.

Specifically, for v_{max} and some arbitrary external flux $J_x = [v]_i = [R_1, ..., J_1, ..., J_x, ..., J_h]$:

$\sum_{k=1}^{n} m_k * b_k = v \leq v_{max}$

$\sum_{k=1}^{n} m_k * b_k * \Delta t \leq v_{max} * \Delta t$

$\sum_{k=1}^{n} \{m_k * \Delta t\} * b_k \leq v_{max} * \Delta t$

$\sum_{k=1}^{n} t_k * b_k \leq v_{max} * \Delta t$

$J_x * \Delta t = \sum_{k=1}^{n} t_k * [b_k]_i \leq [v_{max}]_i * \Delta t$

Invoking the $\overline{Q}$ variables, it is known that $\overline{Q} = 0$ upon the start of every "run" $l_1 \Rightarrow l_2, ..., l_{n-1} \Rightarrow l_n$ and $\overline{Q} = v\Delta t$ at the end of the "run", immediately before $l_n \Rightarrow l_1$. Thus, if the upper bound for J_x is violated and $[v_{max}]_i * \Delta t < [v]_i * \Delta t$, there must exist some time t_Φ in some location l_k, $1 \leq k \leq n$, where $[\overline{Q}_{t_\Phi - dt}]_i < [v_{max}]_i * \Delta t < [\overline{Q}_{t_\Phi + dt}]_i$.

Thus, it becomes necessary to restrain $v * \Delta t$ (and by extension, $[v]_i * \Delta t$) by restricting $\overline{Q}$. This is done by adding transitions from each l_k, $1 \leq k \leq n$ to l_1, all labeled γ, whereupon $[\overline{Q}]_i > [v_{max}]_i * \Delta t$, the transition is forced to l_1 and the "run" is prematurely terminated with the reset $\overline{Q} = 0$. Thus, the final value of $v * \Delta t$ that can be realized by any sequence of transitions $l_1 \Rightarrow l_2, ..., l_n \Rightarrow l_1$ with nondeterminis-

tically chosen transition times $t_1, ..., t_n$ is guaranteed to respect v_{max}. A "premature" transition from $l_k \Rightarrow l_1$ at time t_Φ should still be considered a full "run" and comprise a legitimate realization of $v * \Delta t$ because it is equivalent to the "run" given by $t_1, ..., t_\Phi, 0, ..., 0$, with elapsed time of zero spent in each successor location.

Lower bounds from v_{min} for any arbitrary J_x, $1 \leq x \leq h$ can be established likewise.

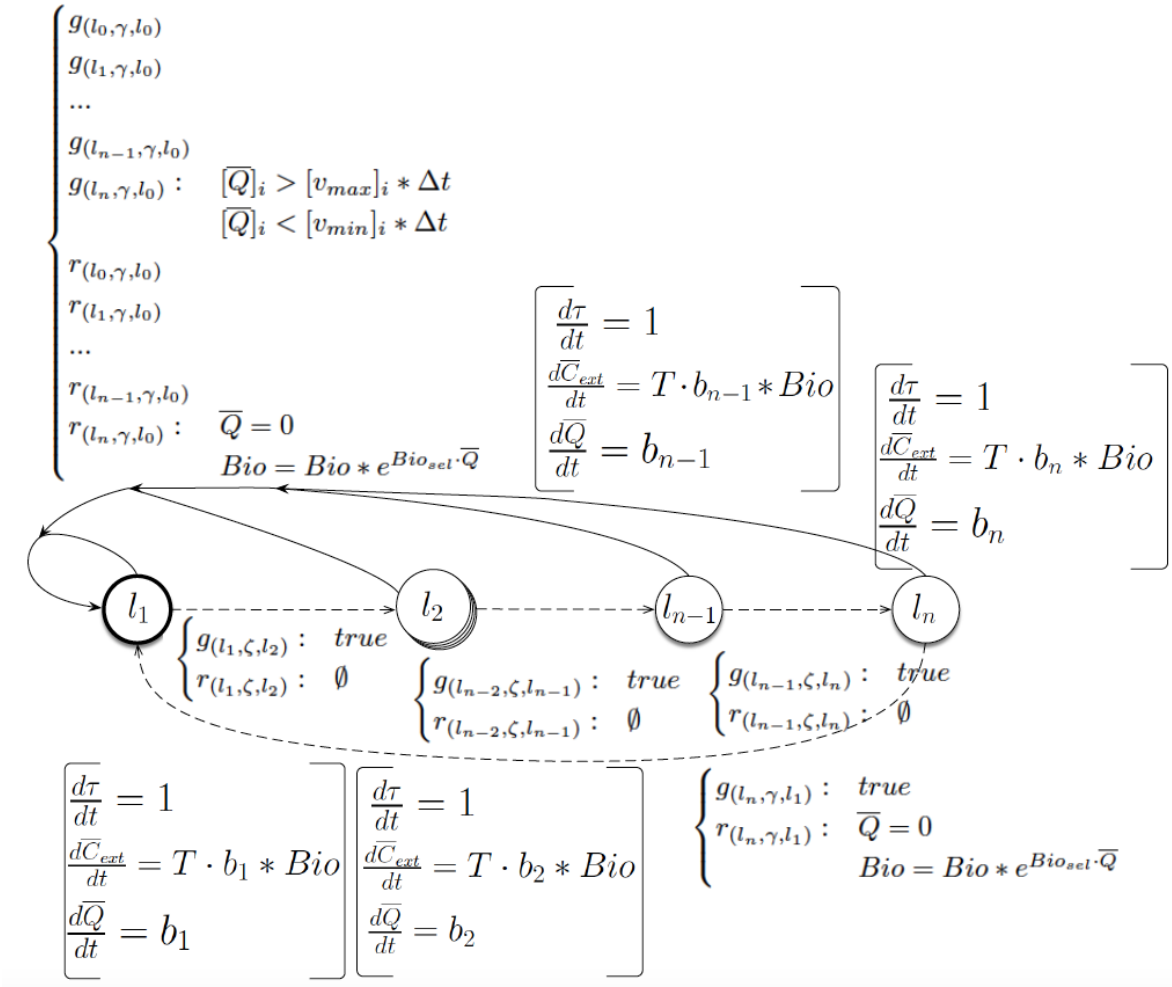


Figure 5.5: Output of Algorithm 2 with Flux Bounds and Biomass Accumulation

5.1.4 Replacing Nondeterminism with Deterministic Control

The benefits to maintaining the nondeterministic formulation are plentiful. Particularly, when one in the early stages of research, the ability to explore all possible (while

phenotypically sound) trajectories is necessary to understand the full metabolic capacity of a microbe. This is particularly useful when analyzing co-cultures, whose synergistic metabolism often exhibits nuanced interactions that can give rise to unexpected behavior over time. In addition, safety analysis can only be properly performed when considering the entire reachable set, or an overapproximation thereof. If only a handful of trajectories result in toxic metabolite secretion, it is preferable (and potentially much less costly) to reveal those scenarios during *in silico* verification than run the risk of toxicity with live microbes. After-the-fact failure analysis also requires exhaustive state-space exploration, as the trajectory realized by the system did not equate to the trajectory predicted. Nevertheless, replacing the nondeterminism with a deterministic control mechanism is necessary if one wishes to simulate the realistic behavior of a microbe (e.g. what would be observed *in vivo*) instead of performing theoretical analysis of the total metabolic capacity.

The nondeterministic transitions between a location l_k and a successor location l_{k+1} allow the automaton to indirectly represent any flux vector $v = \sum_{k=1}^{n} b_k * m_k$ by selecting transition times $[t_1, t_2, ..., t_n] = [\Delta t * m_1, \Delta t * m_2, ..., \Delta t * m_n]$. For each discrete time step of size Δt, which is represented in the automata by a forward "run" from l_1 to l_n, a specific v should be chosen according to some criterion. In other words, the *unforced* transitions that emanate from each location l_k to successor location l_{k+1} (or l_1 in the case of l_n) would be replaced by *forced* transitions with guard conditions of the most general form $G_k(\overline{C}_{ext}) > 0$. It is necessary to make the guard conditions explicit functions of the concentration levels while excluding time itself – the microbe under consideration adjusts its behavior based on the concentrations in the environment, but it not capable of perceiving time nor engaging in temporal reasoning.

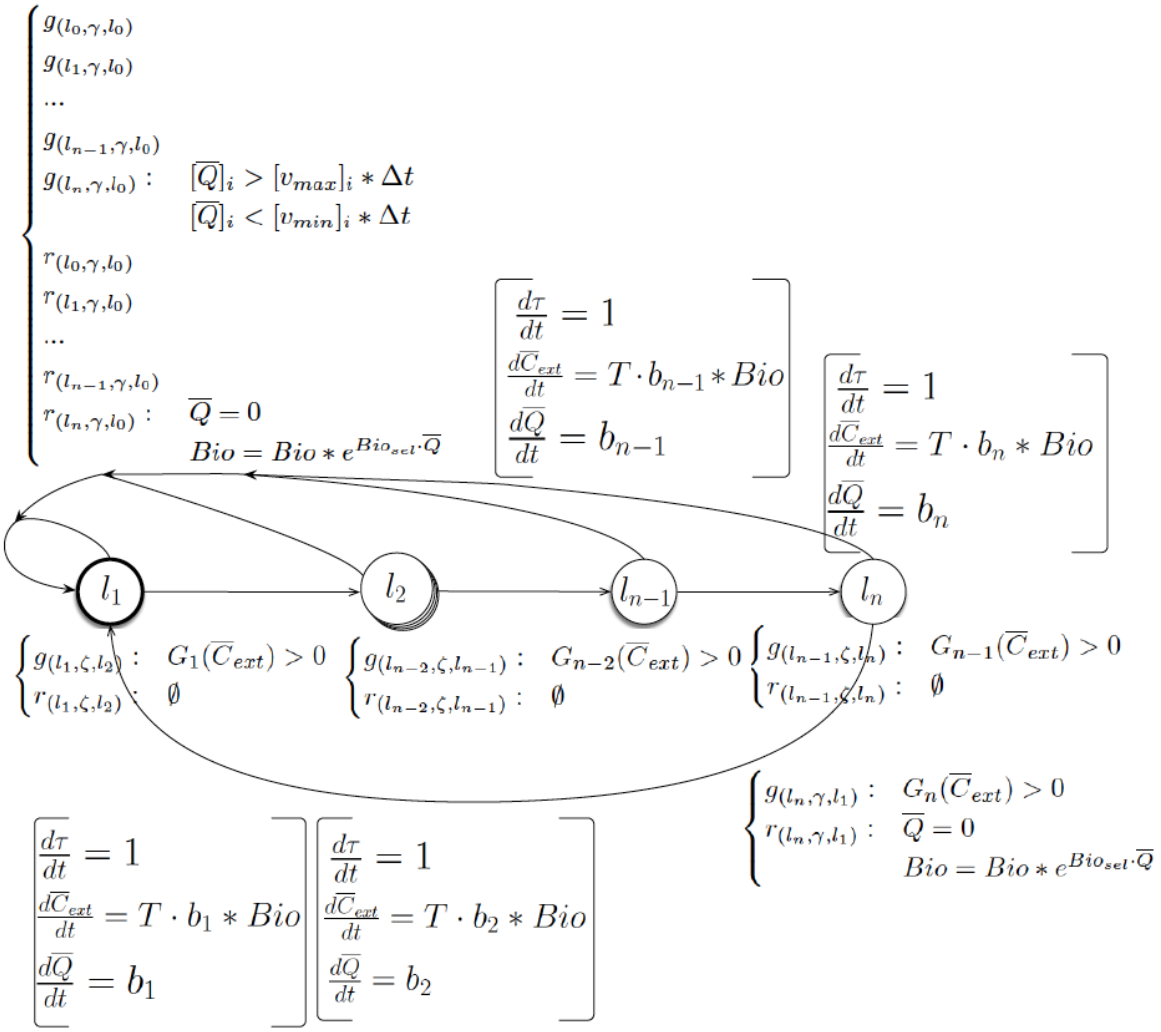


Figure 5.6: Adapting the Automaton to use Deterministic Transitions

It has been shown that biologically relevant criteria for selecting fluxes include the optimal growth objective.

Thus, control functions $G_k(\overline{C}_{ext})$, $1 \leq k \leq n$ should be chosen for each nondeterministic transition such that the resulting deterministic trajectory is optimal (or near-optimal) with respect to an objective function. Formally, this defines an optimal control problem on the automaton. This is a well-studied problem but solving it in the general case lies outside the direct scope of this work. However, the optimal control scenario is important to consider at a high level in order to build and represent

a realistic microbial model. Specifically, in the case of Chiu's model organism, the control mechanism is already known, so the nondeterminism can be neatly replaced by a deterministic criterion without hassle. This is detailed further in Chapter 6.

5.1.5 Co-Culture as Composition of Automata

There are two routes that can be taken to construct an automaton that represents a co-culture. The first is to model the locations, transitions and dynamics present in the co-culture directly, through careful inspection of the behavior of the combined system. This is time-consuming, prone to error, and potentially costly; if any change in co-culture composition is desired (say the user wants to replace one organism for another), the new co-culture must be modelled from the ground-up. The second option is to build a sound modelling framework for individual organisms (as has been done) and define a way to combine arbitrary individual automata into a co-culture automata. This can be achieved using an operation that will be called *inner* (or *dynamic*) *composition*. This builds upon the conventional composition operation for hybrid automata by combining the dynamics for shared variables while respecting the dynamics for exclusive variables. In addition, it synchronizes transitions based on the event labels.

Composition for hybrid automata can be defined in several ways, so *inner composition* will be used to refer to the one that has the following formal definition:

Define the two input automata $\mathbb{H}_1 = (X_1 \cdot L_1, I_1, \Sigma_1, Y_1, F_1, T_1, G_1, R_1)$ and $\mathbb{H}_2 = (X_2 \cdot L_2, I_2, \Sigma_2, Y_2, F_2, T_2, G_2, R_2)$. Let the sets X_1 and X_2 be vectorized as $\overline{X}_1$ and $\overline{X}_2$. Let the vector field F_1 be defined for all locations $l \in L_1$ as $d\overline{X}_1/dt = \overline{f}_l(\overline{X}_1, t)$. Likewise, let the vector field F_2 be given as as $d\overline{X}_2/dt = \overline{f}_l(\overline{X}_2, t)$ for all $l \in L_2$.

Construct a vector from $X_3 = X_1 \cup X_2$ and label it $\overline{X}_3$. Denote a new set of locations $L_3 = L_1 \cdot L_2$, wherein each $l \in L_3$ can be referenced as $(l_1|l_2)$, $l_1 \in L_1$, $l_2 \in L_2$. Endow each location $l \in L_3$ with the continuous state-space X_3. For each transition

$(l_i, s, l_j) \in T_1$, where s is either ζ or $s \notin \Sigma_2$, create transitions $\{((l_i|l_k), s, (l_j|l_k))\}$, $1 \leq k \leq |L_2|$ between each pair of locations $(l_i|l_k), (l_j|l_k) \in L_3$. Endow each transition with guard condition $G_1(t)$ and resets $R_1(t)$, where $t = (l_i, s, l_j)$.

Likewise, create transitions $\{((l_k|l_i), s, (l_k|l_j))\}$, $1 \leq k \leq |L_1|$, between the pairs $(l_k|l_i), (l_k|l_j) \in L_3$ for each $(l_i, s, l_j) \in T_2$, where s is either ζ or $s \notin \Sigma_1$. Endow each transition with guard condition $G_2(t)$ and resets $R_2(t)$, where $t = (l_i, s, l_j)$.

For $s \in \Sigma_1 \cap \Sigma_2$, $s \neq \zeta$, $(l_i, s, l_j) \in T_1$, $(l_u, s, l_v) \in T_2$, create transition $((l_i|l_u), s, (l_j|l_v))$ with guard condition $G_1((l_1, s, l_j)) \cup G_2((l_u, s, l_v))$ and resets $R_1((l_1, s, l_j)) \cup R_2((l_u, s, l_v))$. Add all of the newly created transitions to a set T_3 while adding the associated guards and resets to sets G_3 and R_3, respectively.

Create new sets $I_3 = I_1 \cup I_2$, $\Sigma_3 = \Sigma_1 \cup \Sigma_2$, $Y_3 = Y_1 \cup Y_2$.

For each location $(l_i|l_j) \in L_3$, define $d\overline{X}_3/dt = [dx_1/dt, dx_2/dt, ..., dx_k/dt, ...dx_{|\overline{X_3}|}/dt] = \overline{f}_{l_i|l_j}(\overline{X_3}, t)$ as follows:

$$x_k/dt = \begin{cases} \overline{f}_{l_i}(\overline{X_1}, t) + \overline{f}_{l_j}(\overline{X_2}, t) & x_k \in X_1 \wedge x_k \in X_2 \\ \overline{f}_{l_i}(\overline{X_1}, t) & x_k \in X_1 \wedge x_k \notin X_2 \\ \overline{f}_{l_j}(\overline{X_2}, t) & x_k \notin X_1 \wedge x_k \in X_2 \end{cases}$$

, where $l_i \in L_1, l_j \in L_2$. Label this new vector field over L_3 as F_3. Together, the automaton $\mathbb{H}_3 = (X_3 \cdot L_3, I_3, \Sigma_3, Y_3, F_3, T_3, G_3, R_3)$ is the result of the composition operation.

This operation can also be written, perhaps more elegantly, in vectorized form:

For each $s_i \in S^{'}$, replace the continuous state-space of X_1 with the "enhanced" state-space X_3. For each $x = X_1[k] \in X_1$, let $dx/dt = \overline{f}(\overline{X_1}, t)[k] = f_k(\overline{X_1}, t)$. For $x \in X_3$ where $x \notin X_1$, $dx/dt = 0$. Hence, $\overline{X}_{s_i}/dt = \overline{f}(\overline{X_3}, t)$. Do the same for each $s_j \in S^{''}$ culuminating in $\overline{X}_{s_j}/dt = \overline{f}(\overline{X_3}, t)$. Hence $\overline{X}_{s_i|s_j}/dt$ is given directly by

$\overline{X}_{s_i|s_j}/dt = \overline{X}_{s_i}/dt + \overline{X}_{s_j}/dt.$

5.1.6 Algorithmically Assembling a Model

A third and final algorithm can be formulated to assemble phenotypically-sound microbial models from the stoichiometry of the metabolic network. Unlike the preliminary model produced by Algorithm 2, these models include the flux bounds and growth mechanism as detailed in the previous sections. Thus, these models are a proper bisimulation of the dFBA models output by Algorithm 1 and do not contain trajectories that are not biologically sound.

Algorithm 3: Transformation of dynamical system $\mathbb{D}$ to hybrid automaton $\mathbb{H}(\mathbb{D}, \mathbb{B})$

Input: A dFBA procedure formulated as the dynamical system

$\mathbb{D} = (S, T, v_{max}, v_{min}, \overline{C}_0, \Delta t, t_{max})$

Result: Hybrid automaton $\mathbb{H}(\mathbb{D}, \mathbb{B})$

Solve $S \cdot v = 0$ to find the bases $B = \{b_1, ..., b_n\}$;

Initialize a hybrid automaton $\mathbb{H}$ where:

    Continuous variables $X = \overline{C}_{ext} \cup \overline{Q} \cup \{\tau\}$;

    Discrete locations $L = \{l_1, l_2, ..., l_n\}$;

    Initial condition $I = (l_0, [\overline{C}_{ext}, \overline{Q}, \tau]) = (l_1, [\overline{C}_0, \overline{0}, 0])$;

for $i = 1; i <= n$ **do**

    Assign the dynamics of l_i as follows:

        $\frac{d\overline{C}_{ext}}{dt} = T \cdot b_i * Bio$;

        $\frac{d\overline{Q}}{dt} = b_i$;

        $\frac{d\tau}{dt} = 1$;

    Add an unforced transition from location l_{i-1} to location l_i with guard condition "true" and label η;

    for $j = 1; j <= |\overline{Q}|$ **do**

        Add a forced transition from location l_i to location l_1 with guard condition $[\overline{Q}]_j > [v_{max}]_j * \Delta t$, label γ, and resets $r_{(l_i,\gamma,l_1)} : \{Bio = Bio * e^{Bio_{sel} \cdot \overline{Q}}, \overline{Q} = \overline{0}\}$;

    end

16.

17. **for** $j = 1; j <= |\overline{Q}|$ **do**

18. Add a forced transition from location l_i to location l_1 with guard condition $[\overline{Q}]_j < [v_{min}]_j * \Delta t$, label γ, and resets $r_{(l_i,\gamma,l_1)} : \{Bio = Bio * e^{Bio_{sel} \cdot \overline{Q}}, \overline{Q} = \overline{0}\}$;

19. **end**

20. **end**

21. Add an unforced transition from location l_n to location l_1 with guard condition "true" and resets $r_{(l_i,\gamma,l_1)} : \{Bio = Bio * e^{Bio_{sel} \cdot \overline{Q}}, \overline{Q} = \overline{0}\}$;

22. **Return** $\mathbb{H}$;

Chapter 6

Applying the Framework

6.1 Initial Automata Models

6.1.1 Organism Automata

Finding the general solution of $S \cdot v = 0$ is the first step in constructing the individual automata for Organism 1 and Organism 2 (as defined in section V - A "Formulation of a Reference Model"). For Organism 1, directly solving $S \cdot v = 0$ renders the basis vectors $b_1 = [1, 0, 0.1, 1, -1, -1, 1, 0]$, $b_2 = [0, 1, -0.2, -3, 0, 3, -3, 2]$. However, these must be modified slightly before they can be directly used in constructing an automaton - since v (with respect to Organism 1) has the form $[R_1, R_2, R_3, R_4, J_1, J_2, J_3, J_4]$ and reactions are taken to be irreversible (with reversible reactions split into pairs of irreversible reactions), each internal reaction R_x, $1 \leq x \leq 4$ should have positive flux.

Thus, the b_2 vector must be modified such that the character of the vector as a structural element of the flux cone is preserved, but each internal flux is made positive. Since multiplying b_2 by a negative scalar does not help (R_2 becomes negative), the only other operation that can be performed on b_2 that keeps the resultant vector within the

flux cone is the addition of a scalar multiple of b_1 to b_2. Specifically, the minimal (magnitude) scalar m_{min} must be found whereby $b_2^* = b_1 * m_{min} + b_2$ obeys $R_x \geq 0$, $1 \leq x \leq 4$. m_{min} is trivially found to be 3 and the resulting $b_2^* = [3, 1, 0.1, 0, -3, 0, 0, 2]$ is used to replace b_2 within B.

While the previous process is performed in an *ad hoc* fashion, the concepts involved (constraining the solution of $S \cdot v = 0$ such that internal fluxes remain positive) are formalized by the theory of extreme pathways, which themselves comprise convex bases that span the flux cone. Since manually examining and modifying bases while ensuring that the flux cone remains structurally sound becomes intractable for any $|B| > 2$, the extreme pathways $X = \{x_1, x_2, ..., x_n\}$ are preferred to the naive bases for any model more complex than Chiu's toy model. Indeed, the set $B = \{b_1, b_2^*\}$ can be analytically justified in its usage by observing that the minimal pathways they represent directly correspond to the two basic metabolic pathways present in Organism 1. The b_1 vector represents the pathway that consumes A_{env} and C_{env} by means of J_1 and J_2 and produces Y_{env} through J_3. This is known to be the preferred pathway for the organism. The b_2^* vector represents the pathway that consumes A_{env} by means of J_1 and produces W_{env} through J_4 as a metabolite. This alternative pathway is taken when C_{env} is absent. It is also observable that this pathway is less efficient – it consumes three A_{env} for the same amount of biomass accumulation (through R_3) that consuming one A_{env} and one C_{env} results in. Thus, given the *a posteriori* knowledge of how Organism 1 should function, the usage of B suffices for all practical purposes.

Solving $S \cdot v = 0$ for Organism 2 is a simpler case, rendering $B = \{[1, 0.02, -1, -1, 1]\}$, which needs no further modification as R_5 and R_6 are both positive. In the same analytic process as was performed for Organism 1, the b_1 vector has a clear metabolic interpretation – a single pathway exists that consumes A_{env} and B_{env} through J_5 and J_6, respectively, and produces C_{env} by means of J_7. This is expected and consistent

with known behavior of Organism 2. Thus, each location in the automata models of Organism 1/2 represents a distinct metabolic state, as shown in Fig 6.1 and Fig 7.6.

The bases as previously given, together with the full stoichiometry (internal as well as uptake/secretion) of each organism are transformed into hybrid automata by means of the Algorithm 2 procedure. Biomass accumulation and proper flux bounding are then added, according to the stoichiometry and bounds of each organism. While the $\overline{Q}$ variables should, in theory, be fully inclusive on the flux values such that it realizes $v_i * \Delta t$ upon $l_n \Rightarrow l_1$, $\overline{Q}$ has been restricted to the fluxes that are relevant for enforcing bounds (the uptake reactions) or accumulating biomass. This has been done to declutter the automata diagrams by removing superfluous equations as well as making the simulations significantly more efficient. Each additional variable has a multiplicative effect on the size of the reachable set – the solver is not intelligent enough to determine that a point in the state space is the "same" as its neighbor that only varies in one (superfluous) variable, so redundant work is done when exploring.

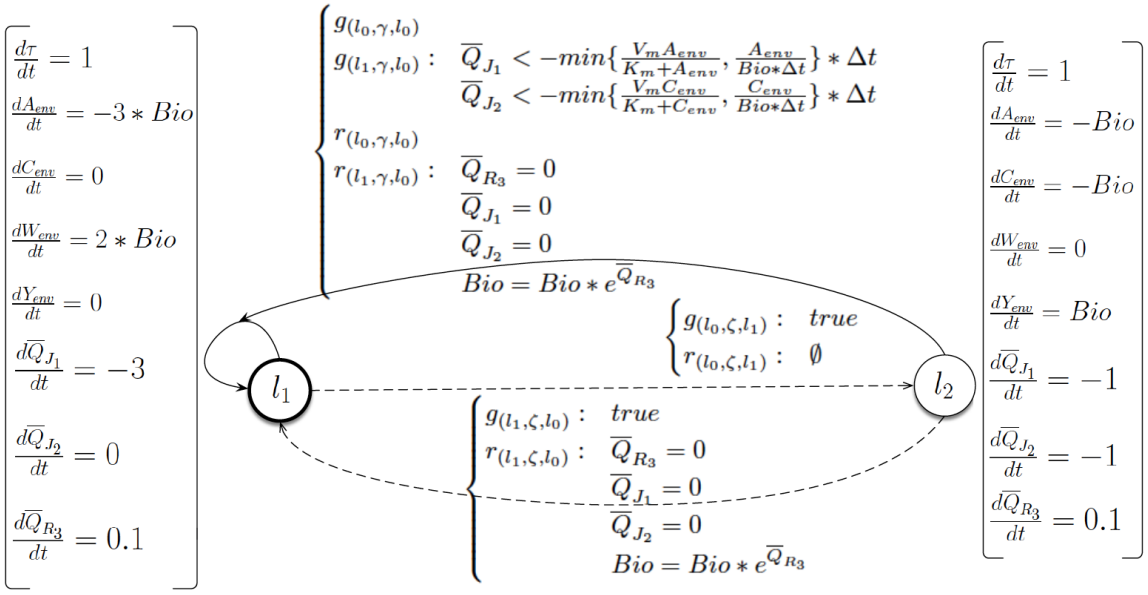


Figure 6.1: Automata representation of Organism 1

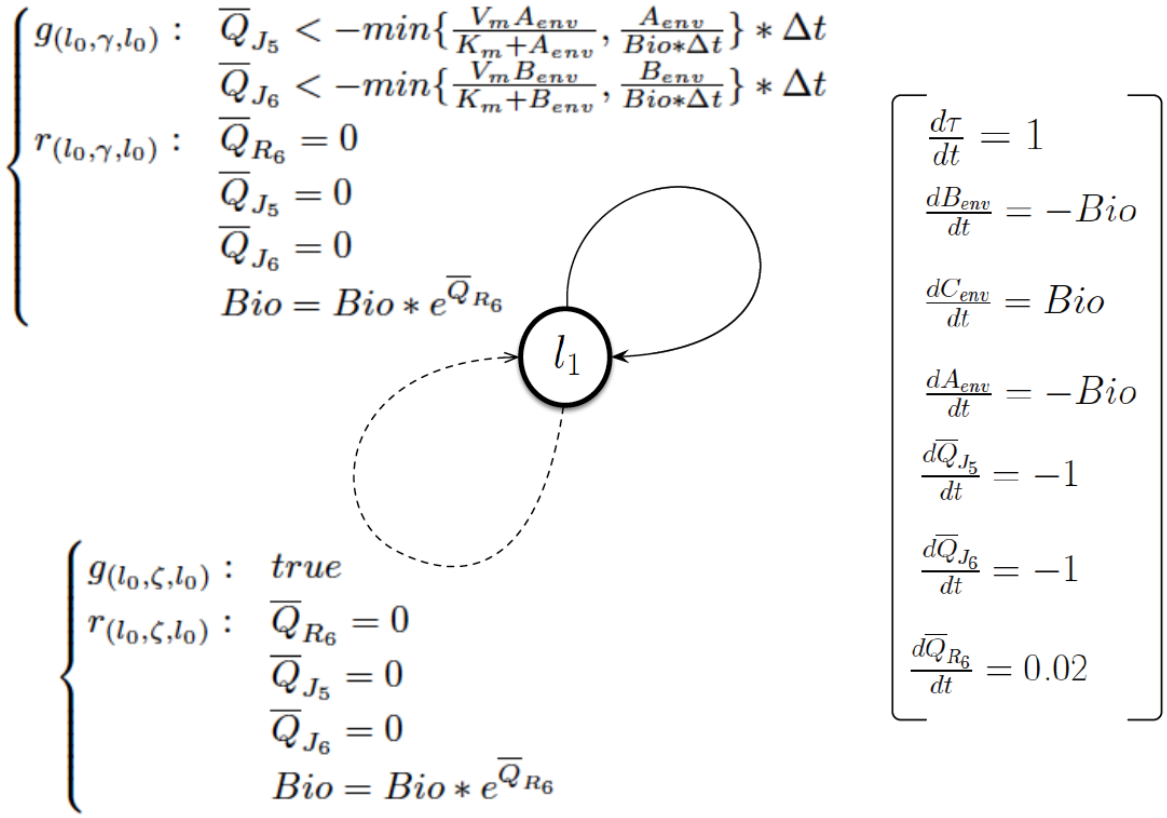


Figure 6.2: Automata representation of Organism 2

6.1.2 Deterministic Control of Model Organisms

For Organism 1 (Fig 6.3), the control mechanism is known and can be used to replace the non-deterministic transitions between l_1 and l_2. The transition from l_1 to l_2 should be forced when the concentration of C_{env}, the metabolite that activates the preferred pathway, rises above some detection threshold ϵ. The transition from l_2 back to l_1 should be forced when the concentration falls back below the threshold (as a result of uptake). The automata corresponding to Organism 2 (Fig 6.4) only has one location, so the non-determinism can be removed entirely. The deterministic transition, while somewhat superfluous, is necessary to ensure that the biomass is adequately updated.

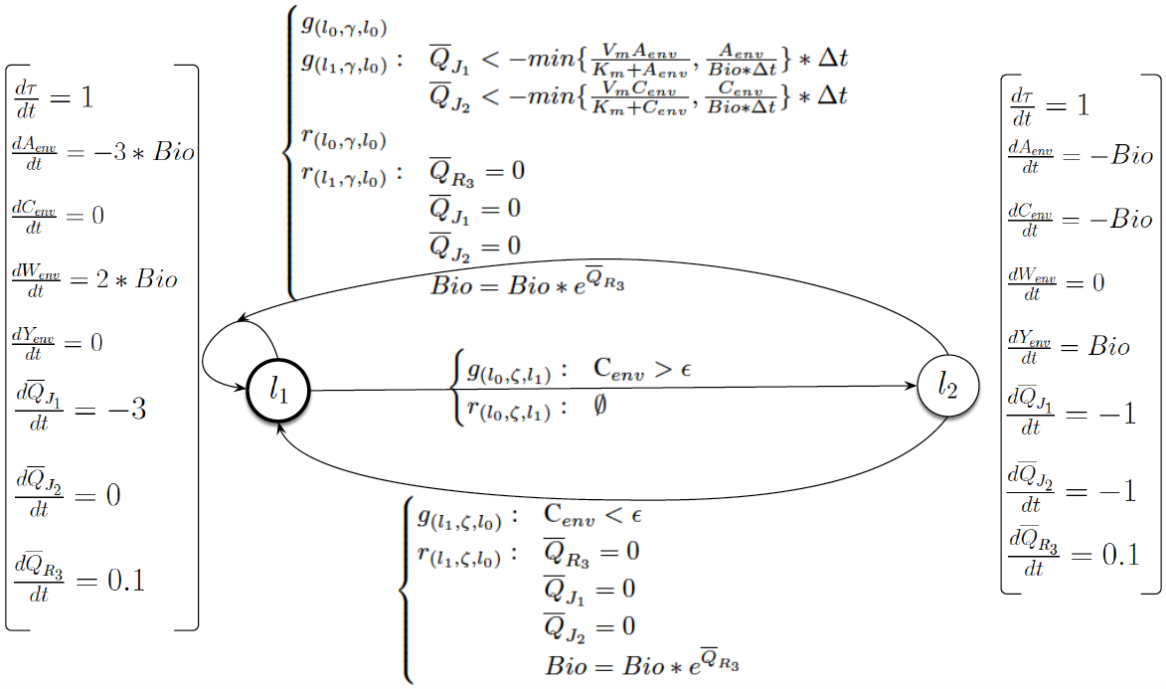


Figure 6.3: Organism 1 with Control Mechanism

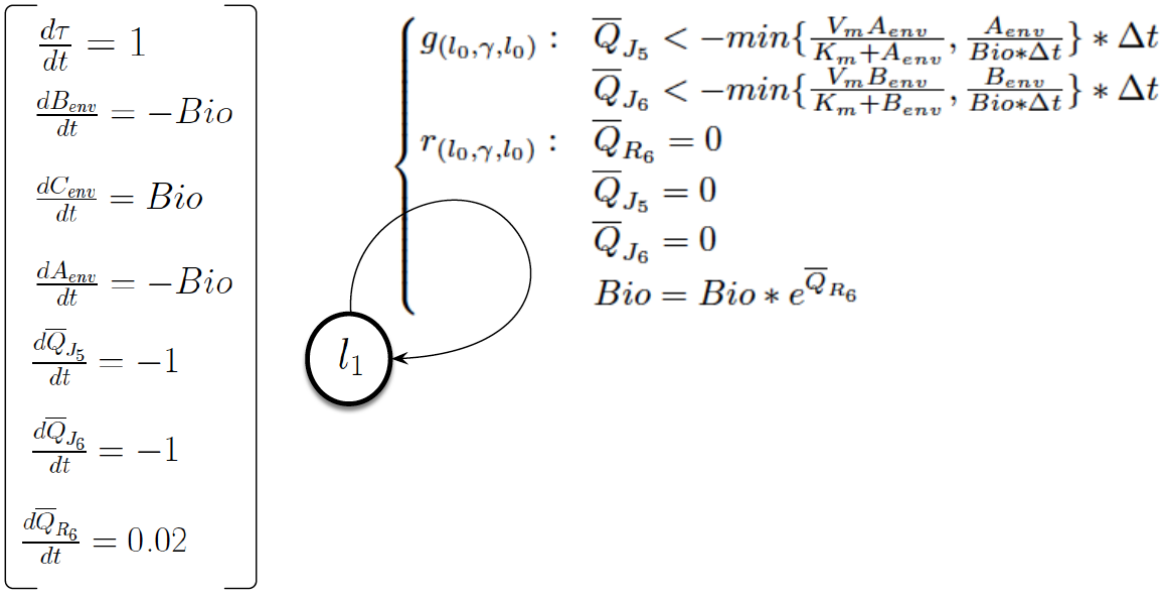


Figure 6.4: Organism 2 with Control Mechanism

6.1.3 Co-Culture Automaton

The inner composition operation can be applied to the prior two automata to construct a model of the co-culture. However, a small change needs to be made before that operation can be performed successfully – the biomass accumulation variable Bio (not formally an internal metabolite) should be made distinct between the two organisms so the composition operation does not treat them as a common variable. Denote the two new variables Bio_1 and Bio_2 and update the resets accordingly. No further preprocessing is needed as the dynamics of $\overline{C}_{ext}$ will be properly combined and the $\overline{Q}$ variables are disjoint between the organisms.

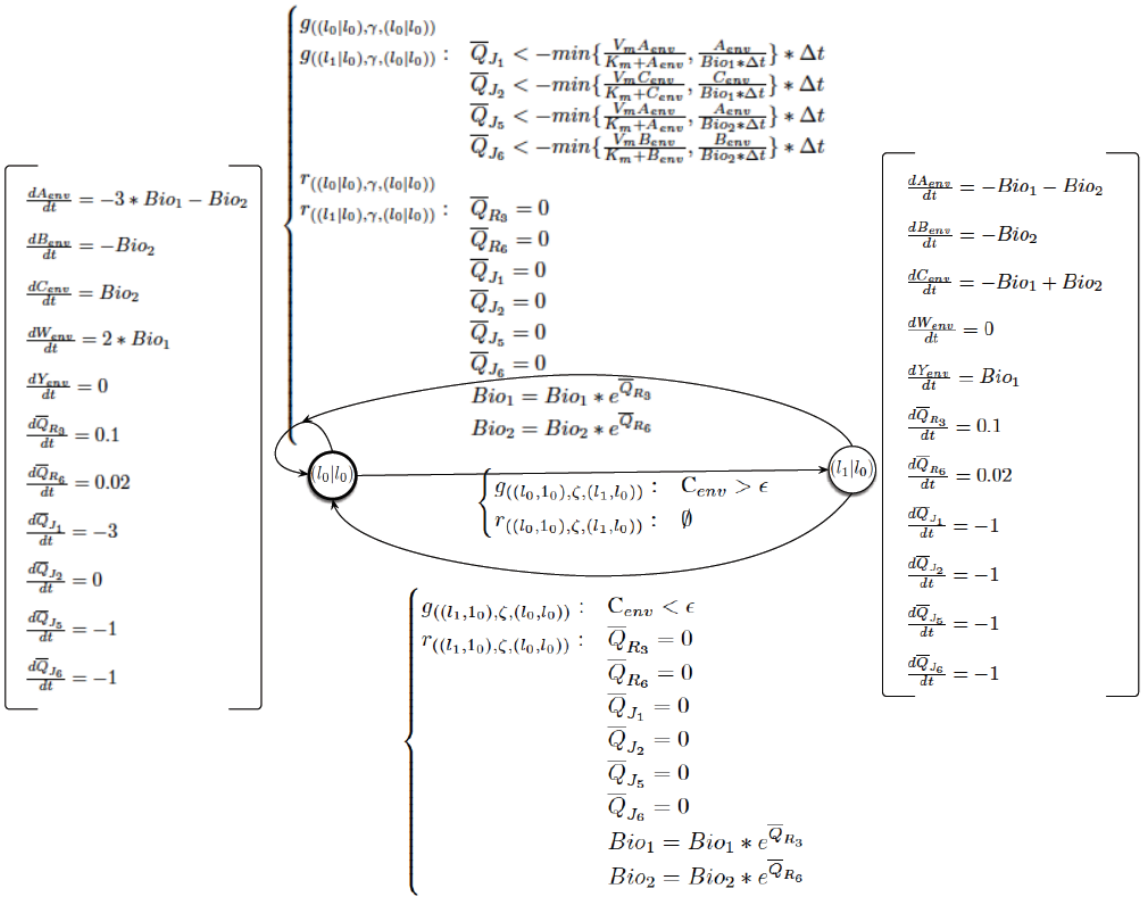


Figure 6.5: Co-Culture with Control Mechanism

6.2 Simulation Environment Setup

6.2.1 Automata Implementation in Ariadne

6.2.2 Automata Variations

6.2.3 Simulation Parameters

6.2.4 Simulation Output, Data Gathering, and Plotting

6.3 Experimental Results

6.3.1 Reachable Set Computation

6.3.2 Qualitative Behavior

Upon formulating the previous three models using the libraries *Ariadne* provides, a comparable simulation (Fig. 6.6) to the one in Chiu's work was run to generate experimental results. We use the time scale $t_{max} = 20$ with $\Delta t = 0.01$. The automata under consideration are entirely deterministic, so only one run is necessary for each (no averaging of results is necessary).

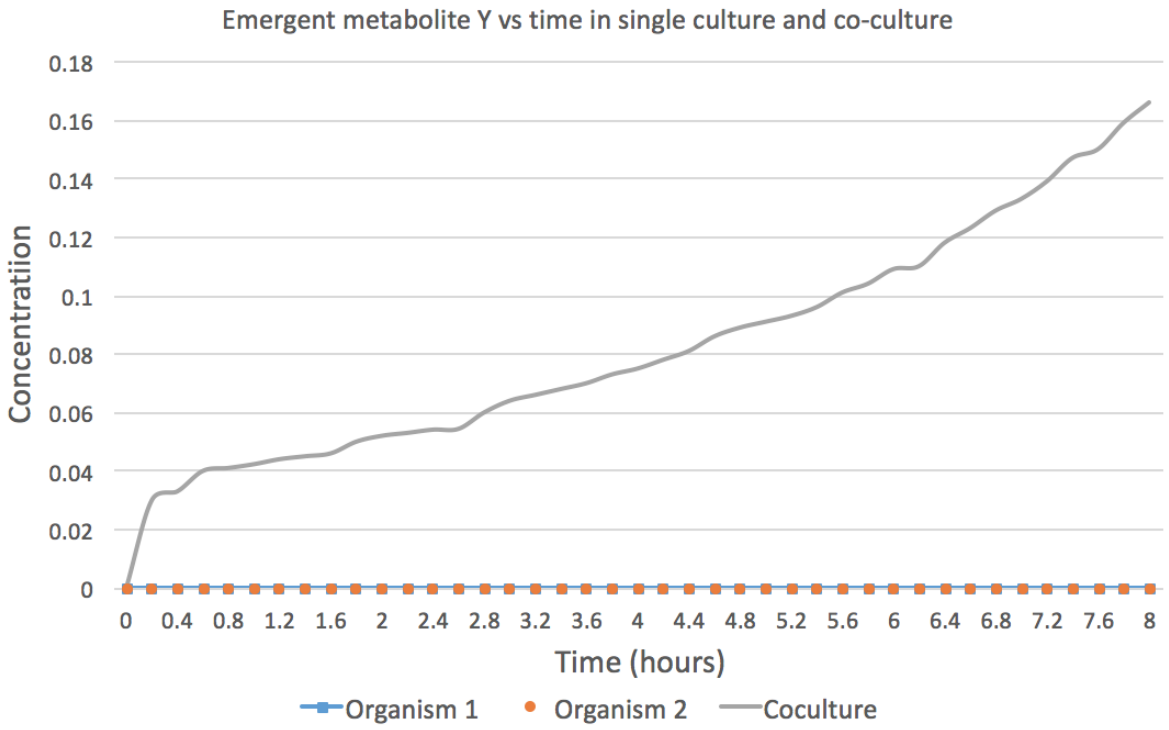


Figure 6.6: Levels of emergent metabolite Y in single cultures and co-culture

6.3.3 Formal Verification of Emergent Metabolism

Chapter 7

Systems Biology Research with Hybrid Automata

While directly solving the equation $Sv = 0$ to receive the set of bases $B = \{b_1, b_2, ..., b_n\}$ is sufficient to satisfy the constraints of the framework and build a phenotypically-sound microbial model, this naive formulation is not optimized for the experimentation and analysis tasks demanded by research practitioners. As discussed in Chapter 2, there are a number of methods from the systems biology literature that decompose the steady-state flux space into a set of convex bases. These methods are preferred because they offer phenotypically-relevant context that allows the researcher to interpret the functional significance of the individual bases within the broader metabolic network. When building a framework for experimentation and analysis with the ultimate goal of furthering explainability of metabolic processes, such additional context is invaluable to both the model itself (insofar as it offers additional properties that can be formalized, traced, and verified) and to the researcher (by providing a helpful elucidation of the biological significance of otherwise opaque formalisms). Thus, the modelling framework

with hybrid automata can be enhanced by replacing the set of bases $B = \{b_1, b_2, ..., b_n\}$ with the convex bases rendered by one of the systems biology approaches for decomposing the flux cone. The theoretical implications as well as practical concerns raised when employing these approaches is detailed further below. A brief discussion of the advanced analytical capability provided by the enhanced model, with details of how it could be harnessed in a research pipeline, is also included.

As a reference point, the general form of an organism automaton (as discussed in Chapter 5) is given below. Modifications to this form, brought on by the replacement of the $B = b_1, b_2, ..., b_n$ vectors, will be presented in the following sections. The nondeterministic form is essential to the generality of the model. The distinct benefits (as well as potential drawbacks) of replacing the nondeterminism with a defined control mechanism will be a key point of discussion in a section to follow. As a side note, the growth mechanism given is the same as Chiu's formulation, but it need not necessarily be. A more straightforward growth mechanism would also suffice without loss of generality.

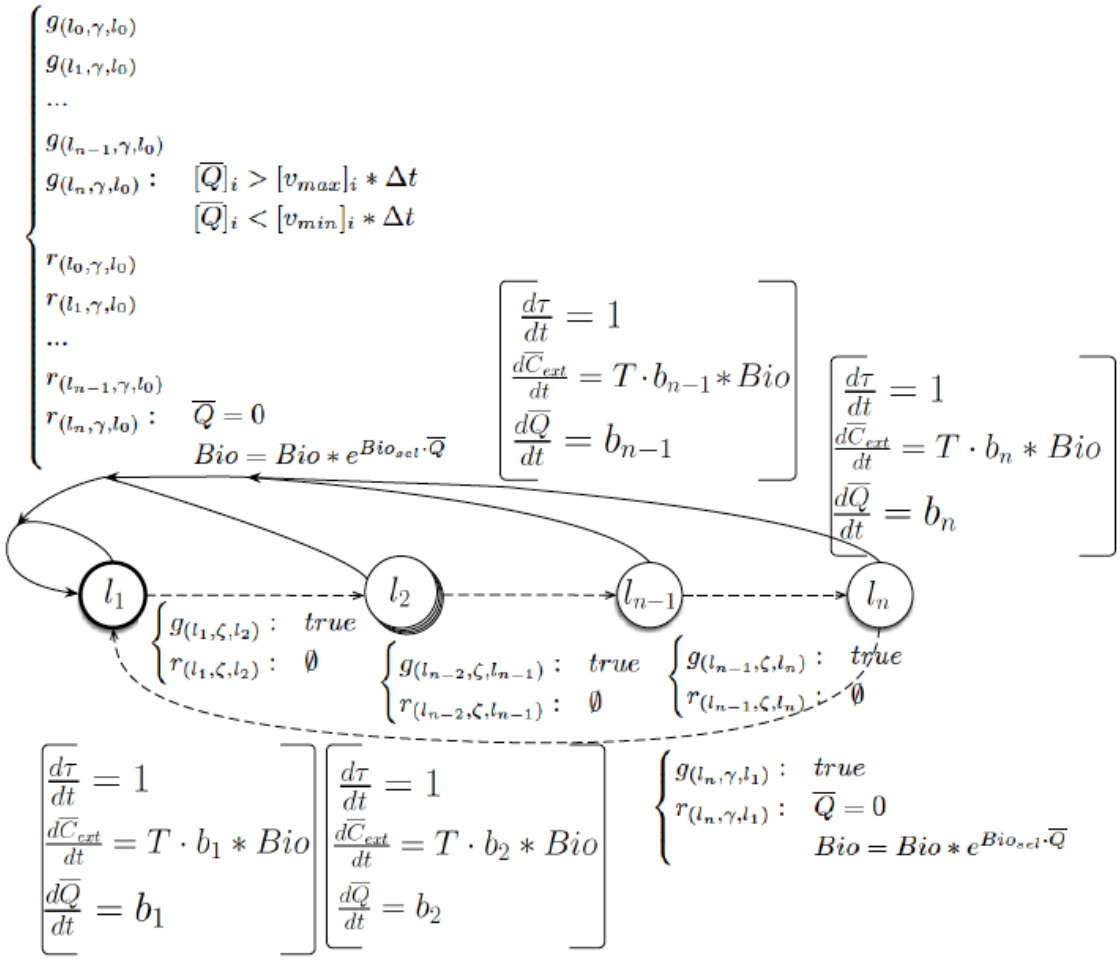


Figure 7.1: General form of a microbe automaton using naive bases

7.1 Extreme Pathways to Regulatory Mechanisms

7.1.1 Singular Value Decomposition and Eigenpathways

As discussed in Chapter 2, the extreme pathways are a set of convex bases that fully characterize the phenotypically viable solution space. However, without a corresponding kinetic model, the extreme pathways alone cannot be used to directly elucidate the underlying regulatory mechanisms that give rise to metabolic processes. Singular value decomposition can be applied to the matrix of extreme pathways (or *expas*),

wherein the left singular vectors of the SVD are the *eigenpathways* of the metabolic network. The eigenpathways have been shown to characterize the physiologically relevant metabolic states that would otherwise be computed with a detailed kinetic model. Furthermore, in the case of the red blood cell metabolism, the first five (in order of significance) eigenpathways out of a total of 23 characterize the relevant regulatory mechanisms that underlie shifts in metabolic behavior. Thus, the use of eigenpathways instead of the raw expas implicitly reduces the dimensionality of the viable flux space. In the case of the red blood cell metabolism, only five degrees of freedom are needed to characterize 91% of the relevant fluxes within the flux cone.

For an automata model of a microbe, the set of bases $B = \{b_1, b_2, ..., b_n\}$ can be replaced by the set of eigenpathways $E = \{e_1, e_2, ..., e_m\}$ rendered by the SVD of the expas of S, without loss of generality. Furthermore, E can be restricted to the significant subset $E_k = \{e_1, e_2, ..., e_k\}, k \leq m$, such that the relevant flux space is still effectively characterized but the degrees of freedom of the model (and locations in the automaton) are greatly reduced. This renders the model more compact and hence computationally efficient while also adding additional interperability for the researcher designing and running experiments.

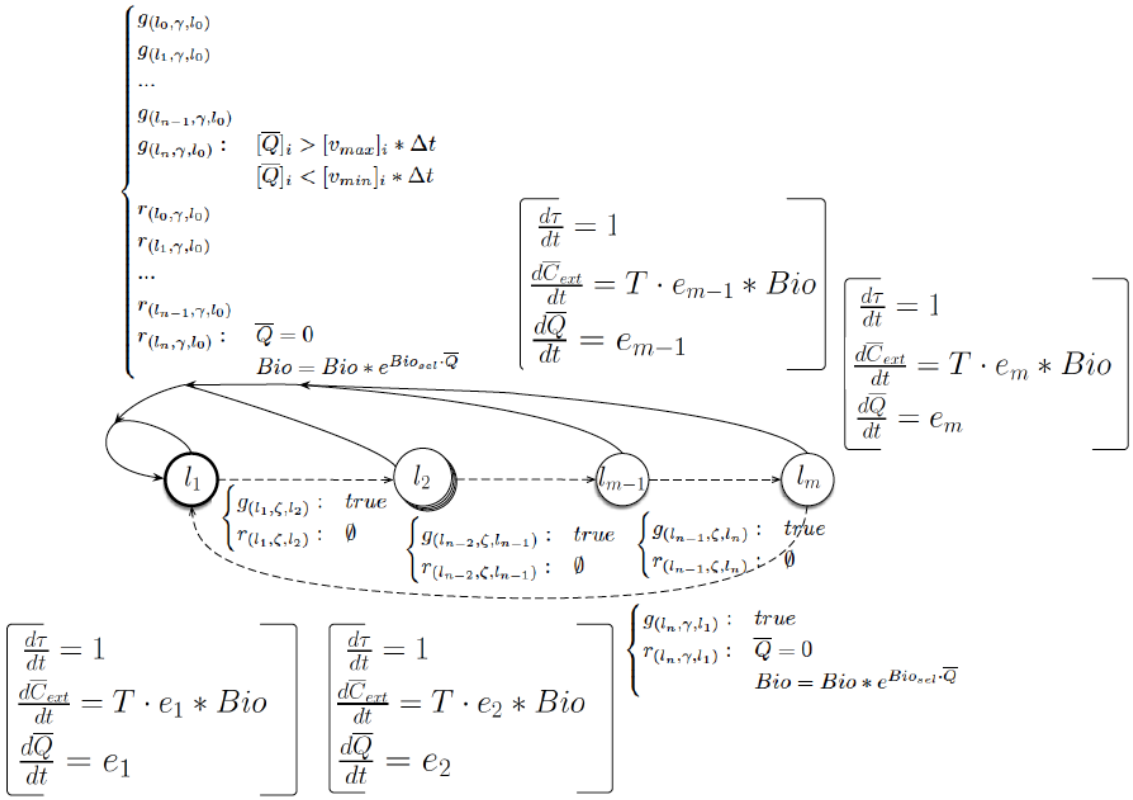


Figure 7.2: Microbe automaton as parameterized by the eigenpathways

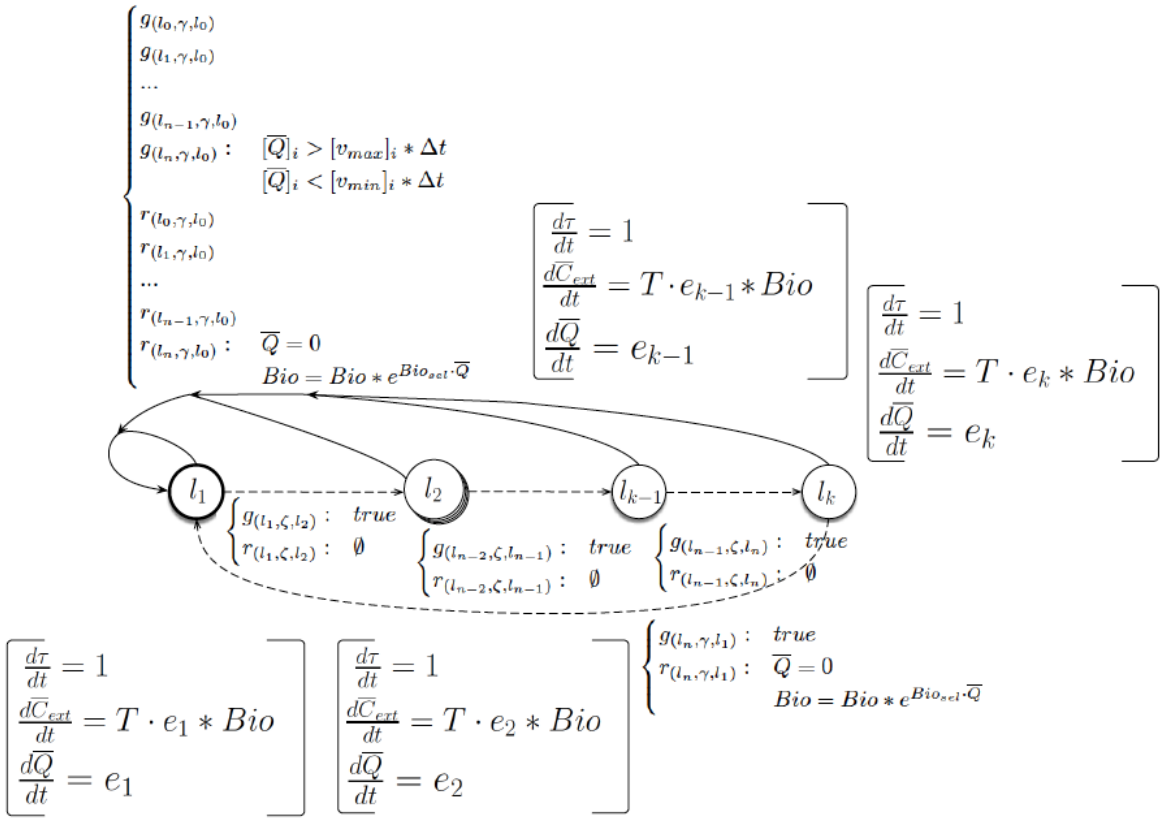


Figure 7.3: Microbe automaton as parameterized by E_k. Generally, $k << m$, rendering the model significantly more computationally efficient.

7.1.2 Nondeterministic Regulatory Dynamics

With the parameterization $E_k = \{e_1, e_2, ..., e_k\}$ that directly relates locations in the automaton to regulatory modes, more robust experiments can be run than the naive parameterization alone supports. Experiments of the following nature can be posed and answered:

1. Do the trajectories that lead to failure under some safety analysis scenario always include or exclude a specific regulatory mode?

2. Is emergent metabolism (in a co-culture automata) dependent on some regulatory mode in one (or both) of the microbes being reached?

3. Is a specific sequence of regulatory shifts a necessary precursor to some metabolite to be produced?

4. Will a specific regulatory shift always result in some metabolite being produced?

Answering these questions involves exhaustive exploration of the solution space, using the nondeterminism present in the model to guarantee the realization of all valid trajectories. The newly established relationship between locations and regulatory modes allows the regulatory shifts taken to be embedded in the trajectory itself, along with the concentrations of metabolites. The regulatory mode represented by location l_i is r_i.

Within each forward "run", a sequence of regulatory shifts is realized as the locations are traversed. Since the time spent in each location corresponds to the net time in a regulatory mode, only consider locations that have $\Delta t_i > 0, 1 \leq i \leq k$. Thus, upon the transition $l_n \Rightarrow l_1$, the set $R = \{r_i \mid l_i \in L,\ \Delta t_i > 0,\ 1 \leq i \leq k\}$ represents the regulatory modes entered while realizing some flux vector v. This set of regulatory modes is implicitly ordered as the forward transition topology forces an ordering onto the locations l_i. Thus, a sequence of forward "runs" realizing the trajectory $\mathbb{D}(\overline{V} = [v_1, v_2, ..., v_n])$ induces a sequence of sets $[R_1, R_2, ..., R_n]$. As the sets themselves are implicitly ordered, this sequence can be "flattened" into a one-dimensional vector of regulatory modes $\overline{R} = [r_1, r_2, ...]$. This is the sequence of regulatory shifts taken when realizing $\mathbb{D}(\overline{V})$.

Thus, when exploring the solution space by enumerating all $\{\mathbb{D}(\overline{V})\}$, the subset of $\{\mathbb{D}(\overline{V})\}$ that satisfies some reachability condition (emergent metabolite production, failed safety analysis property) can have queries regarding regulatory shifts evaluated over $\{\overline{R}\}$.

The above necessitates the development of a formalization that unifies reachability

queries in the continuous space with properties to evaluate over the location-space induced by all satisfiable trajectories.

7.1.3 Formalizing Regulatory Queries

Continuous property satisfied ⇒ Query over sequence of locations

7.1.4 From Optimal Control to Regulatory Mechanisms

The newly established correspondence between locations and regulatory mechanisms makes the optimal control problem on organism automata significantly more symbolically meaningful. When locations come to represent regulatory modes, solving for the control functions that deliver a near-optimal trajectory with respect to some objective translates into finding the environmental conditions that trigger a regulatory shift. This presents a tremendous leap forward in the analytical ability that the automata framework provides for analyzing and explaining metabolic behavior. When using the naive bases, the optimal control problem could be formulated and solved on the automaton, allowing the researcher to observe the environmental shifts underlying emergent metabolic capacity, but provided no additional interperability nor explainability over the dFBA approach. The transition behavior was a "black box" from the perspective of the researcher. Incorporating eigenpathways (and hence regulatory mechanisms) into the framework allows the unification of steady-state metabolic simulation with detailed analysis of regulatory mechanisms *without* requiring any sort of kinetic model that the existing approaches integrating regulation with dFBA rely upon. With this capability in mind, the environmental conditions along with the relevant regulatory shifts that trigger emergent behavior in co-culture models can be studied and isolated. Furthermore, complex metabolic engineering scenarios (with one or more microbes present)

can be formulated wherein the regulatory shifts observed can be compared to a well-studied reference model to verify the soundness of the *in silico* behavior. Like the other components of this modelling framework, knowledge of the stoichometry is all that is needed to build and verify microbial models, with no assumptions made about the kinetic mechanisms underlying the metabolic behavior.

The OC problem is also computationally simpler to solve when parameterizing the model with eigenpathways – the reduction in degrees of freedom results in k total locations instead of the original m/n. Hence, only $G_h(\overline{C}_{ext})$, $1 \leq h \leq k$ deterministic control functions must be found.

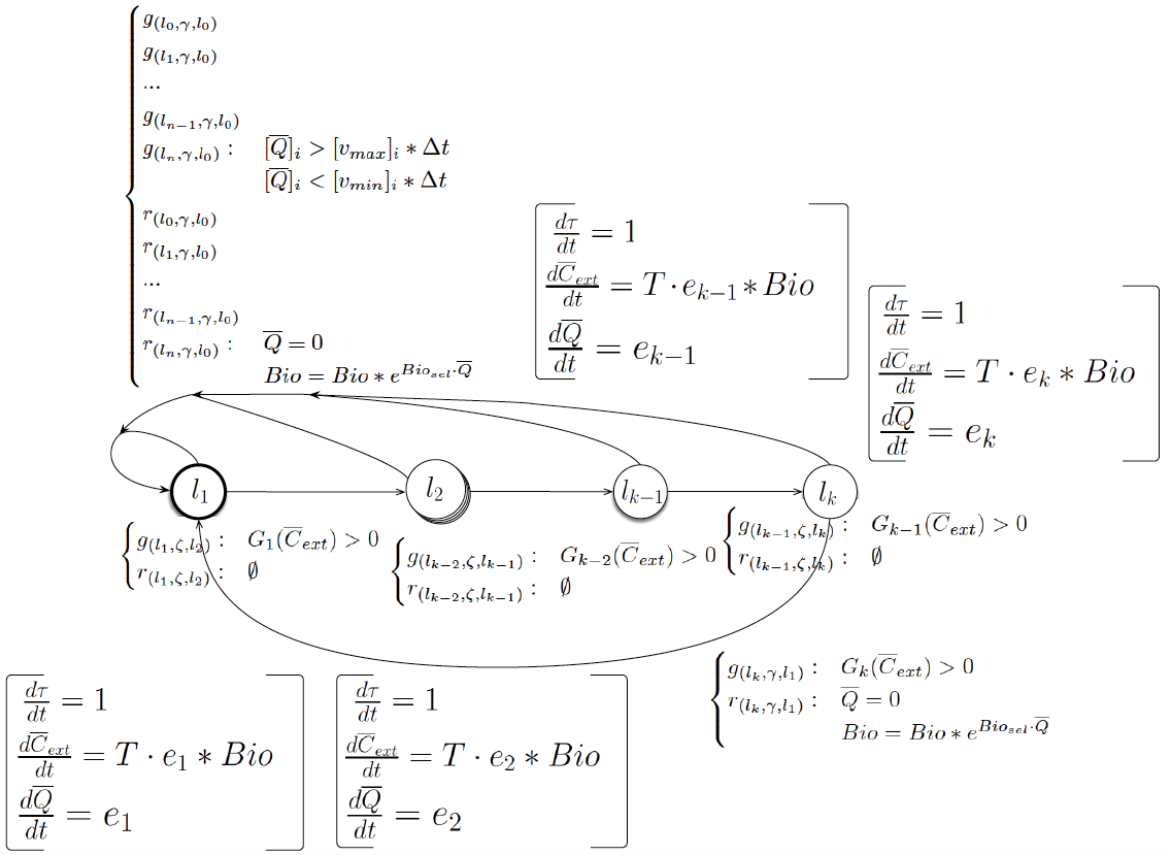


Figure 7.4: Microbe automaton with E_k parameterization and deterministic control functions.

Before considering this a realistic model of regulatory shifts, a question must be

raised regarding the transition topology of this model versus the traditional model of regulatory behavior. Specifically, the most general model of regulation allows for transitions between each pair of regulatory modes as in the diagram that follows:

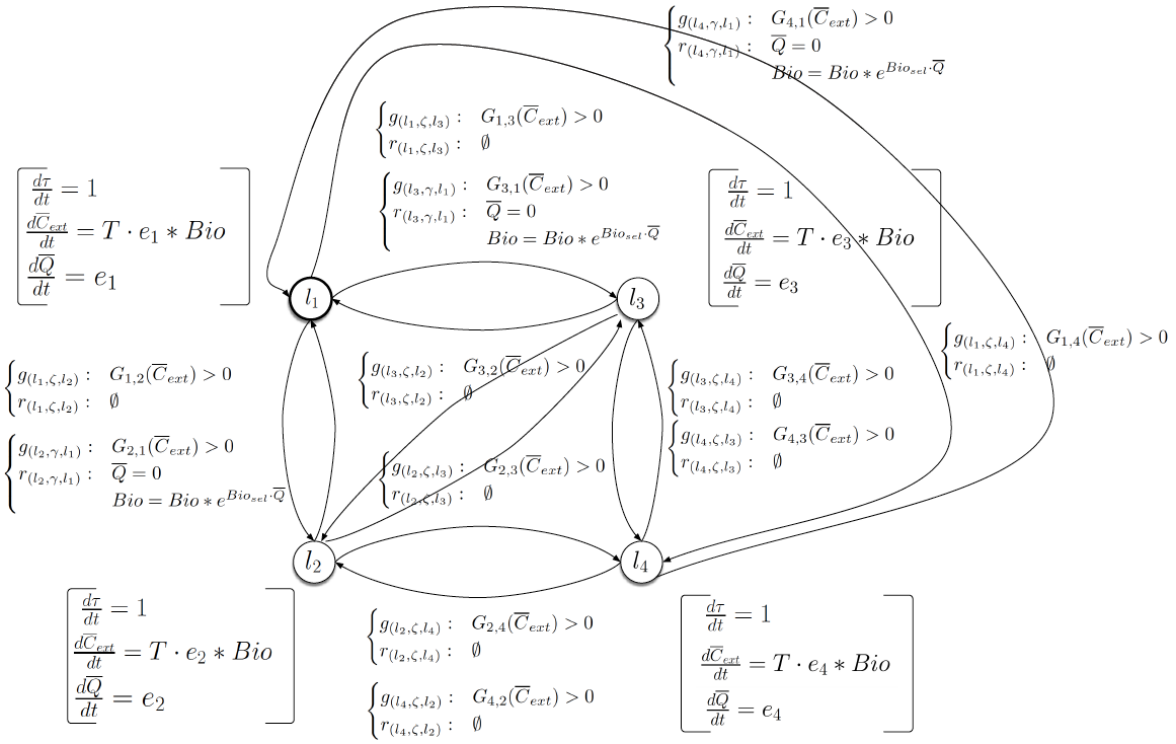


Figure 7.5: Automaton representing cellular regulation as generally understood. Each regulatory mode can transition to any other regulatory mode. $k = 4$ has been chosen but k is arbitrary. Transitions to l_1 driven by $[\overline{Q}]_i > [v_{max}]_i * \Delta t$ and $[\overline{Q}]_i < [v_{min}]_i * \Delta t$ have been omitted for clarity but would be present in the model.

A similar model is featured in (paper) as part of their regulatory-kinetic hybrid automaton model of cellular metabolism. It is not immediately clear if the flux-based model (with eigenpathways) that features only forward transitions between locations $l_1, ..., l_k$ fully encapsulates the same range of behaviors. For this model to suffice, a bisimulation must be drawn between the two.

In the case of the model with only forward transitions along $l_1, l_2, ..., l_k$, the control mechanism for some location l_i must be established that the transition is triggered

whenever another location is desired, regardless of the proper originating location for such a transition in the fully connected automaton. Consider the case where $k = 4$. If the current location is l_3 and the transition activated is $l_3 \rightarrow l_2$, in the fully connected automaton, that transition will be triggered directly by the control mechanism $G_{3,2}(\overline{C}_{ext})$. However, the model with only forward transitions does not have a direct transition between location l_3 and l_2 and hence no control mechanism can properly be defined over it. The path $l_3 \rightarrow l_4 \rightarrow l_1 \rightarrow l_2$ instead must be taken and the control mechanism $G_{3,2}(\overline{C}_{ext})$ that signifies the environmental conditions inducing the regulatory shift must govern all three of those transitions. Furthermore, the elapsed time in l_4 and l_1 must be zero - a regulatory mode r_i is only considered enabled if $\Delta t_i > 0$. To represent a regulatory shift from r_3 directly to r_2, the transitions $l_4 \rightarrow l_1$ and $l_1 \rightarrow l_2$ must fire instantaneously upon entering the originating state.

To solve the above dilemma, control functions along the forward transitions (count-

ing l_4 to l_1 at the end of a "run" as a forward transition) should be defined as follows:

$$\begin{aligned}
G_1(\overline{C}_{ext}) = {} & G_{1,2}(\overline{C}_{ext}) > 0 \;||\; G_{1,3}(\overline{C}_{ext}) > 0 \;||\; G_{1,4}(\overline{C}_{ext}) > 0 \;||\\
& G_{2,3}(\overline{C}_{ext}) > 0 \;||\; G_{2,4}(\overline{C}_{ext}) > 0 \;||\\
& G_{3,2}(\overline{C}_{ext}) > 0 \;||\; G_{3,4}(\overline{C}_{ext}) > 0 \;||\\
& G_{4,2}(\overline{C}_{ext}) > 0 \;||\; G_{4,3}(\overline{C}_{ext}) > 0\\
G_2(\overline{C}_{ext}) = {} & G_{2,1}(\overline{C}_{ext}) > 0 \;||\; G_{2,3}(\overline{C}_{ext}) > 0 \;||\; G_{2,4}(\overline{C}_{ext}) > 0 \;||\\
& G_{1,3}(\overline{C}_{ext}) > 0 \;||\; G_{1,4}(\overline{C}_{ext}) > 0 \;||\\
& G_{3,1}(\overline{C}_{ext}) > 0 \;||\; G_{3,4}(\overline{C}_{ext}) > 0 \;||\\
& G_{4,1}(\overline{C}_{ext}) > 0 \;||\; G_{4,3}(\overline{C}_{ext}) > 0\\
G_3(\overline{C}_{ext}) = {} & G_{3,1}(\overline{C}_{ext}) > 0 \;||\; G_{3,2}(\overline{C}_{ext}) > 0 \;||\; G_{3,4}(\overline{C}_{ext}) > 0 \;||\\
& G_{1,2}(\overline{C}_{ext}) > 0 \;||\; G_{1,4}(\overline{C}_{ext}) > 0 \;||\\
& G_{2,1}(\overline{C}_{ext}) > 0 \;||\; G_{2,4}(\overline{C}_{ext}) > 0 \;||\\
& G_{4,1}(\overline{C}_{ext}) > 0 \;||\; G_{4,3}(\overline{C}_{ext}) > 0\\
G_4(\overline{C}_{ext}) = {} & G_{4,1}(\overline{C}_{ext}) > 0 \;||\; G_{4,2}(\overline{C}_{ext}) > 0 \;||\; G_{4,3}(\overline{C}_{ext}) > 0 \;||\\
& G_{1,2}(\overline{C}_{ext}) > 0 \;||\; G_{1,3}(\overline{C}_{ext}) > 0 \;||\\
& G_{2,1}(\overline{C}_{ext}) > 0 \;||\; G_{2,3}(\overline{C}_{ext}) > 0 \;||\\
& G_{3,1}(\overline{C}_{ext}) > 0 \;||\; G_{3,2}(\overline{C}_{ext}) > 0
\end{aligned}$$

More generally, this formulation can be written for some automaton with k indeterminate as:

$$G_i(\overline{C}_{ext}) = \bigcup_{\substack{n,m=1 \\ n \neq m \\ m \neq i}}^{k} G_{n,m}(\overline{C}_{ext}) > 0$$

It must be shown that, given the control functions as defined above, some environ-

mental trigger for a regulatory shift in the fully-connected automaton will induce the same regulatory behavior in the forward-connected automaton. Showing full regulatory sequence equivalence is not necessary, as behavioral equivalence upon the same set of environmental triggers will invariably lead to regulatory sequence equivalence between the two automata.

For the fully-connected automaton, $G_{3,2}(\overline{C}_{ext}) > 0$ in l_3 will directly fire a transition from l_3 to l_2 representing the regulatory shift from r_3 to r_2. No further elucidation is necessary in this case.

However, in the forward-connected automaton, $G_{3,2}(\overline{C}_{ext}) > 0$ in l_3 will trigger $G_3(\overline{C}_{ext})$ firing $l_3 \rightarrow l_4$. Upon entering l_4, the relation $G_{3,2}(\overline{C}_{ext}) > 0$ still holds and triggers $G_4(\overline{C}_{ext})$ firing $l_4 \rightarrow l_1$. Again, $G_{3,2}(\overline{C}_{ext}) > 0$ is still valid upon entering l_1 and triggers $G_1(\overline{C}_{ext})$, firing $l_1 \rightarrow l_2$. Finally, upon reaching l_2, $G_{3,2}(\overline{C}_{ext}) > 0$ holds but is not a component element of $G_2(\overline{C}_{ext})$, leading the system to stay in l_2 for some $\Delta t_2 > 0$. Since the elapsed time in both l_4 and l_1 is 0, the sub-sequence of regulatory shifts induced is simply $r_3 \rightarrow r_2$.

7.2 From Genome-Scale Models to Hybrid Automata

Genome-scale models are the state-of-the-art in modern systems biology research, providing the ability to study real-world organisms *in silico* by providing a tremendously detailed description of the internal stoichiometry. High-definition genome sequencing techniques are employed to build accurate stoichiometric models on the order of tens of thousands of individual reactions. However, a number of computational issues must be rectified before genome-scale models can be fully integrated into the hybrid automata framework. The tabular approach for finding extreme pathways quickly becomes computationally intractable on a system of thousands of linear equations. Thus,

an alternative method for computing extreme pathways should be devised such that the structure of the flux cone is preserved while the convex bases themselves are computable in polynomial time.

7.2.1 Sampling the Flux Space

Paullson and co. have devised a method to compute extreme pathways by repeated statistical sampling of the flux space.

7.2.2 A Genome-Scale Analytical Pipeline

Putting the previous sections together allows the full pipeline for experimentation on genome-scale models to be formalized.

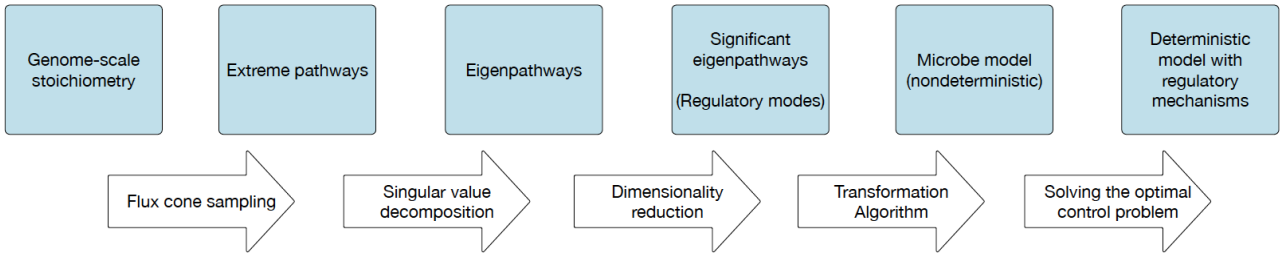


Figure 7.6: Model generation pipeline from genome-scale stochiometry.

This pipeline can be run to completion or stopped at the stage where it generates nondeterministic models, which are particularly useful for safety analysis and full state-space exploration.

The algorithm that follows gives the detailed procedure for genome-scale model generation:

Algorithm 4: Transformation of dFBA trajectory $\mathbb{D}(\overline{V})$ to hybrid automaton $\mathbb{H}(\mathbb{D}, \overline{V})$

1. **Input:** The trajectory $\mathbb{T} = (\mathbb{D}, \overline{V} = [v_1, v_2, v_3..., v_n]) = \mathbb{D}(\overline{V})$;
   Result: Hybrid automaton $\mathbb{H}(\mathbb{D}, \overline{V})$ that follows trajectory $\mathbb{D}(\overline{V})$;
2. Initialize a hybrid automaton $\mathbb{H}$ where:
3. Continuous variables $X = \overline{C}_{ext} \cup \{\tau\}$;
4. Discrete locations $L = \{l_1, l_2, ..., l_n\}$;
5. Initial condition $I = (l_1, [\overline{C}_0, 0])$;
6. Assign the dynamics of l_1 as:
7. $\frac{d\overline{C}_{ext}}{dt} = T \cdot v_1 * Bio$;
8. $\frac{d\tau}{dt} = 1$;
9. **for** $i = 2; i <= n$ **do**
10. Assign the dynamics of l_i as follows:
11. $\frac{d\overline{C}_{ext}}{dt} = T \cdot v_i * Bio$;
12. $\frac{d\tau}{dt} = 1$;
13. Add a forced transition from l_{i-1} to l_i with guard condition $\tau > \Delta t$ and reset $\tau = 0$;
14. **end**
15. **Return** $\mathbb{H}$;

7.3 Other Convex Basis Parameterizations

7.3.1 Elementary Mode Analysis

Alternate convex basis representations of the flux cones should also be considered as lending interperability to the model. One of the most promising of these is the set of elementary modes $M = \{m_1, m_2, ...m_m\}$ for a given stoichiometric matrix S.

Chapter 8

Conclusions and Future Work

8.1 Conclusions

Several important conclusions can be derived from the previous – namely, (a) a computational framework has been developed that is theoretically at least as powerful as dFBA (b) uses standard hybrid automata representation without invoking DAEs or other unorthodox constructs to complicate simulation and analysis (c) produces empirical results under numerical simulation that are comparable to the ones one would expect to see from dFBA as applied to the same model system, and (d) is generically parameterized such that automata can be produced given only the stoichiometry and parameters of simulation, with little knowledge of the mechanisms that govern the organism required. Thus, a highly extensible framework for *in silico* microbial experimentation has been developed. Applicable theory that has not directly incorporated in this work but should be considered for immediate exploration includes: Optimal Control Theory, which will allow us to handle the generic case (for organisms with unknown control mechanisms), solving the optimal control problem is necessary to derive deterministic transition equations for the automaton.

Another topic to consider for future exploration is extreme pathway analysis in which replacing the naive bases $B = \{b_1, ..., b_n\}$ of S with the set of extreme pathways $E = \{e_1, ..., e_n\}$ maintains the generality of the framework and the structure of the flux cone while ensuring that positivity constraints (and other biological constraints) will always be satisfied.

8.2 Future Work

8.2.1 Genome-Scale Model Analysis

8.2.2 Solving the Optimal Control Problem

8.2.3 Differential Dynamic Logic

Bibliography

[1] S. A. Becker, A. M. Feist, M. L. Mo, et al. Quantitative prediction of cellular metabolism with constraint-based models: the cobra toolbox. *Nature protocols*, 2:727–738, 2007.

[2] M.B. Biggs, G.L. Medlock, G.L. Kolling, and J. A. Papin. Metabolic network modeling of microbial communities. *Wiley Interdisciplinary Reviews: Systems Biology and Medicine*, 7(5):317–334, 2015.

[3] K. Brenner, L. You, and F. H. Arnold. Engineering microbial consortia: a new frontier in synthetic biology. *Trends in Biotechnology*, 26:483–489, 2008.

[4] C. H. Chang, P. Graf, D. M. Alber, et al. Photons, photosynthesis, and high-performance computing: challenges, progress, and promise of modeling metabolism in green algae. *Journal of Physics: Conference Series*, 125(1):012048, 2008.

[5] H. C. Chiu, R. Levy, and E. Borenstein. Emergent biosynthetic capacity in simple microbial communities. *PLoS Computational Biology*, 10:e1003695, 2014.

[6] R.A.B. Fazzini, M. J. Preto, A. C. Poucas Quintas, A. Bielecka, and V. A. P. Martins Dos Santos. Consortia modulation of the stress response: proteomic analysis of single strain versus mixed culture. *Environmental Microbiology*, 12:2436–2449, 2010.

[7] J. A. Fuhrman. Microbial community structure and its functional implications. *Nature*, 459:193–199, 2009.

[8] S. Greenblum, P. J. Turnbaugh, and E Borenstein. Metagenomic systems biology of the human gut microbiome reveals topological shifts associated with obesity and inflammatory bowel disease. *Proceedings of the National Academy of Sciences of the United States of America*, 109:594–599, 2012.

[9] J. Handelsman, J. Tiedje, L. Alvarez-Cohen, M. Ashburner, I.K.O. Cann, E.F. Delong, W.F. Doolittle, C.M. Fraser-Liggett, Adam Godzik, J.I. Gordon, et al. *The new science of metagenomics: revealing the secrets of our microbial planet*. National Academies Press, 2007.

[10] T.J. Hanly. *Dynamic Modeling of Synthetic Microbial Consortia to Optimize the Co-fermentation of Glucose and Xylose*. PhD thesis, University of Massachusetts Amherst, 2013.

[11] T.J. Hanly and M. A. Henson. Dynamic flux balance modeling of microbial co-cultures for efficient batch fermentation of glucose and xylose mixtures. *Biotechnology and Bioengineering*, 108:376–85, 2011.

[12] T.J. Hanly and M.A. Henson. Dynamic model-based analysis of furfural and HMF detoxification by pure and mixed batch cultures of *S. cerevisiae* and *S. stipitis*. *Biotechnology and Bioengineering*, 111:272–284, 2014.

[13] L. Heirendt, S. Arreckx, T. Pfau, et al. Creation and analysis of biochemical constraint-based models using the cobra toolbox v.3.0. *Nat Protoc*, 14:639–702, 2019.

[14] et al. Hickert, L.R. Ethanogenic fermentation of co-cultures of candida shehatae hm 52.2 and saccharomyces cerevisiae icv d254 in synthetic medium and rice hull hydrolysate. *Bioresource Technology*, pages 508–514, 2013.

[15] J. P. Kaye, R. L. McCulley, and I. C. Burke. Carbon fluxes, nitrogen cycling, and soil microbial communities in adjacent urban, native and agricultural ecosystems. *Global Change Biology*, 11:575–587, 2005.

[16] M. Kuyper, M. J. Toirkens, J. A. Diderich, et al. Evolutionary engineering of mixed-sugar utilization by a xylose-fermenting strain. *FEMS Yeast Research*, 5:925–934, 2005.

[17] R.E. Ley. Obesity and the human microbiome. *Current Opinion in Gastroenterology*, 26:5–11, 2010.

[18] Lee R. Lynd, Paul J. Weimer, Willem H. van Zyl, and Isak S. Pretorius. Microbial cellulose utilization: Fundamentals and biotechnology. *Microbiology and Molecular Biology Reviews*, 66(3):506–577, 2002.

[19] T. Moutinho Jr. and et. al. Novel co-culture plate enables growth dynamic-based assessment of contact-independent microbial interactions. *PLOS ONE*, 2017.

[20] J.D. Orth, I. Thiele, and B.O. Palsson. What is flux balance analysis? *Nat Biotechnol*, 28:245–248, 2010.

[21] E. J. O'Brien, J. M. Monk, and B. O. Palsson. Using genome-scale models to predict biological capabilities. *Cell*, 161:971–987, 2015.

[22] E. Palmqvist and B. Hahn-Hägerdal. Fermentation of lignocellulosic hydrolysates. i: inhibition and detoxification. *Bioresource Technology*, pages 17–24, 2000.

[23] Octavio Perez-Garcia, Gavin Lear, and Naresh Singhal. Metabolic network modeling of microbial interactions in natural and engineered environmental systems. *Frontiers in Microbiology*, 7:673, 2016.

[24] J. Qin, Y. Li, Z. Cai, S. Li, J. Zhu, et al. A metagenome-wide association study of gut microbiota in type 2 diabetes. *Nature*, 490:55–60, 2012.

[25] R. Ramadas and M. Thattai. Flipping DNA to Generate and Regulate Microbial Consortia. *Genetics*, 184(1):285–293, 2009.

[26] F. Salimi, K. Zhuang, and R. Mahadevan. Genome-scale metabolic modeling of a clostridial co-culture for consolidated bioprocessing. *Biotechnol J*, 5:726–738, 2010.

[27] et al. Srilekha Yadav, K. Bioethanol fermentation of concentrated rice straw hydrolysate using co-culture of saccharomyces cerevisiae and pichia stipitis. *Bioresource Technology*, pages 6473–6478, 2011.

[28] N.C. VerBerkmoes, V. J. Denef, R. L. Hettich, and J. F. Banfield. Systems biology: Functional analysis of natural microbial consortia using community proteomics. *Nat Rev Microbiol*, 7(3):196–205, 2009.

[29] Z. Wang, S.A. Danziger, B.D. Heavner, S. Ma, Smith J. J., S. Li, et al. Combining inferred regulatory and reconstructed metabolic networks enhances phenotype prediction in yeast. *PLoS Computational Biology*, 13(5):e1005489, 2017.

[30] J. Zaldivar, J. Nielsen, and L. Olsson. Fuel ethanol production from lignocellulose: a challenge for metabolic engineering and process integration. *Applied Microbiology and Biotechnology*, 56(1-2):17–34, 2001.

Appendix A

Background

Lorem ipsum dolor sit amet, idque apeirian mnesarchum vix ea, usu no mandamus constituam comprehensam, cum cu vero definitionem. Fabulas probatus ne pro, eleifend hendrerit adipiscing at vim. Usu eu facilisis persecuti. Ne has vidit principes efficiantur, has liber euripidis no. Vide sapientem salutandi per at. Eu probo democritum referrentur pri, an vidit simul reprimique nam. Omnium eruditi ex per, elit regione reprimique te eos. Dicam timeam no nam. Tacimates indoctum inciderint ad his, sea fastidii appetere oporteat an. Ad cum movet efficiantur. Sale labitur no vis, complectitur consequuntur usu te. Mel te putant timeam denique. Ne mel vidit ancillae. Ius saperet facilisi an. Eligendi scaevola sea ex, ad recteque cotidieque quo. Te sumo brute eum.

Lorem ipsum dolor sit amet, idque apeirian mnesarchum vix ea, usu no mandamus constituam comprehensam, cum cu vero definitionem. Fabulas probatus ne pro, eleifend hendrerit adipiscing at vim. Usu eu facilisis persecuti. Ne has vidit principes efficiantur, has liber euripidis no. Vide sapientem salutandi per at. Eu probo democritum referrentur pri, an vidit simul reprimique nam. Omnium eruditi ex per, elit regione reprimique te eos. Dicam timeam no nam. Tacimates indoctum inciderint

ad his, sea fastidii appetere oporteat an. Ad cum movet efficiantur. Sale labitur no vis, complectitur consequuntur usu te. Mel te putant timeam denique. Ne mel vidit ancillae. Ius saperet facilisi an. Eligendi scaevola sea ex, ad recteque cotidieque quo. Te sumo brute eum.